基于有限差分的药物烘干规律探究

摘　要

随着医药行业的发展，药材处理的应用显得即为广泛，而干燥工序恰恰是其中至关重要的一道工序。为此，本文基于热风烘干的干燥方式，中药具体的物理性质，建立数学模型探究药材水分浓度以及温度的变化规律，并求解最短烘干时间。

对于问题一，本文通过分析烘房环境，对药材模型进行了可视化处理，再基于**傅里叶导热定律与菲克扩散定律**建立圆柱药材轴对称的水分浓度与温度控制方程模型，并限定了初值条件，施加**对流换热—对传质边界条件**，轴心由对称性取零通量；水分扩散系数 $D(C)$ 按附录 2 经验公式随含水率变化。接下来，采用**有限体积法通量守恒离散**与**全隐式时间推进**：轴心取**四倍系数对称格式**，表面以**半格控制体**将边界条件隐式嵌入，非线性扩散系数 $D(C)$ 滞后线性化。最终通过对所生成的图像的分析，发现了预热平衡阶段，药材温度和水分均呈显著的径向梯度。**热量随时间整体升高且内外温差拉大；水分随时间整体下降且表面失水更快，水分梯度变陡**。整体传热（由表及里）与传质（由内向外）方向相反，药材中心的变化明显滞后于表面。

对于问题二，药材的物理性质 ρ、c_p、k 随含水率变化，水分扩散系数 $D(C,T)$ 同时依赖含水率与温度，温度场与水分场由问题一的独立求解升级为**经物性双向耦合**；烘房环境按“预热平衡—恒温干燥”两阶段驱动，切换时刻 $t_c = 2h$，参数组随阶段切换。我们仍沿用问题一的控制方程，在此基础上将其改进为分段函数，保持其他条件不变。数值求解上，空间采用**有限差分法**，时间采用**显隐混合格式**推进（内部节点显式、表面半格节点隐式）以解除边界刚性限制，界面导热系数采用调和平均、扩散系数采用**线性插值**，同时引入显式格式稳定性预检与严格的质量守恒校核。最终通过插值输出指定位置和时间的温度与水分浓度, 结论与问题一类似。

对于问题三，背景、代码延用问题二的模型与显隐混合数值格式，核心区别在于将仿真时长延伸至烘干完成，引入了**动态事件检测机制**：以药材中心水分浓度首次降至 $0.15kg/kg$ 为判据, 通过线性插值精确定位烘干完成时刻 t_f，解得**烘干时间为 $54.77h$**。

对于问题四，其核心是处理烘干过程中的尺寸收缩（移动边界）。代码采用 **Landau 坐标变换**（ξ=r/R(t)）将移动域转化为固定计算域，控制方程转化为仅含 $R(t)$ 的形式；物性采用附录 4 公式，半径 $R(t)$ 由附件 2 实测数据经 **PCHIP 插值**，表面边界条件相应变换、换热系数 R(t)h。数值上采用问题二类似的解法，基于最小半径进行稳定性预检。以中心水分首次降至 0.15 kg/kg 为终止判据并额外推进 1 小时，同时设置 **regr（回归验证）与 fix4（归因分解）双算例**。最终解得**烘干时间为 $49.45h$**。

最后，针对烘干过程中的物性变化，对含水量下界 C_prop 以及对流换热系数 h、对流传质系数 h_m 进行**灵敏度分析**，考虑了不确定因素之后，最终结果差距不大，表明模型有较好的稳健性。

关键词：　　有限差分; 显隐混合格式; 数值仿真

一、 问题重述

1.1 问题的背景

中药材作为我国传统医学的瑰宝，其药效的发挥与成品的质量稳定性息息相关。在中药材的加工与炮制过程中，干燥是至关重要的一道工序。干燥不仅是为了去除药材中多余的水分以防止霉变、便于储存和运输，更是为了固定药材的有效成分，保证药效。在众多的干燥方式中，热风烘干因其设备相对简单、操作便捷、处理量大而被广泛应用。

热风烘干过程通常包含预热平衡和恒温干燥两个主要阶段。在预热平衡阶段，药材从初始状态进入烘房，受到热风的作用，其表面及内部温度逐渐升高，同时水分开始迁移并蒸发。这一阶段的传热传质涉及热空气与药材表面的对流换热、药材内部的热传导以及水分在浓度梯度和温度梯度驱动下的扩散。

然而，在实际生产中，干燥工艺参数（如热风温度、湿度、流速等）的选取往往依赖经验或传统的“试错法”。若参数选取不当，极易导致干燥效率低下、能源浪费严重，甚至造成药材有效成分流失、色泽改变、质地变差等品质问题。随着计算机技术和数值计算方法的飞速发展，利用数理分析与数值仿真手段来模拟干燥过程已成为解决上述问题的有效途径。通过建立精确的数学模型，可以定量描述药材在干燥过程中的温度场和水分场演变，预测不同工艺条件下的干燥效果，从而为优化干燥工艺、提高产品质量、降低能耗提供科学依据和理论指导。

1.2 问题的提出

针对中药材烘干过程中的传热传质机理及工艺优化需求，本文需依次解决以下四个递进问题：

1. **问题一：预热平衡阶段的状态变化**
   考虑药材为圆柱体（长 25 cm，半径 2 cm），初始温度 28°C，初始水分浓度 2.55 kg/kg。基于附件 1 给定的烘房温湿度时变边界条件及附录 2 中的物性参数，建立预热平衡阶段药材内部温度场与水分浓度场的耦合数学模型。求解该模型，按表 1 和表 2 格式输出特定时刻及特定位置的数值结果。
2. **问题二：全流程烘干过程的连续模拟**
   针对持续 2 ～ 3 天的完整烘干周期，考虑到预热与恒温干燥阶段物性参数的差异性，利用附录 3 提供的经验公式，建立涵温度与水分浓度演变模型。计算并展示前 3 h 内每隔 0.5 h 的数据，同时生成整个过程的数据集。
3. **问题三：基于固定尺寸的干燥终点预测**
   以药材内部各处水分浓度降至 0.15 kg/kg 以下作为干燥完成判据，基于前述模型确定所需的总烘干时长。在论文中按表 5 格式呈现每隔 6 h、空间间隔 0.5 cm 的水分分布概况，并生成完整的水分浓度变化数据。
4. **问题四：变尺寸效应下的干燥时长修正**
   进一步考虑实际烘干过程中因水分流失导致的药材尺寸收缩效应。根据附件 2 确定的收缩规律及附录 4 中的经验公式，修正上述模型以反映动态边界变化，重新确定满足干燥要求所需的时长。在论文中按表 6 格式给出每隔 6 h、空间间隔 0.5 cm

的水分浓度分布，并生成完整的水分浓度变化数据。

二、 问题分析

2.1 问题一的分析

本题聚焦于某特定圆柱形中药材在热风烘干过程中“预热平衡阶段”的温度和水分浓度变化规律。已知该中药材的几何尺寸为长 25 cm、半径 2 cm。在烘干初始时刻（t=0），药材整体处于均匀状态，其初始温度为 28°C，初始干基含水率为 2.55 kg/kg。烘房内的环境条件（温度与湿度）随时间的变化情况已知。

我们需要基于传热学与传质学的基本原理，结合给定的药材物性参数，建立描述该圆柱体药材在预热平衡阶段内部温度分布和水分浓度分布演变的数学模型。我们发现这个问题属于抛物型偏微分方程组的初边值问题（IBVP）——传热方程与扩散方程经 D(C) 弱耦合。

2.2 问题二的分析

物理过程与建模思路问题二与问题一有很多相似的地方，在问题一的基础上，问题二的温度 T 和水分浓度 C 呈现强耦合性特征，这意味着温度的变化会影响水分的扩散速度，而水分的迁移又会改变材料的热物性参数，必须联立求解。

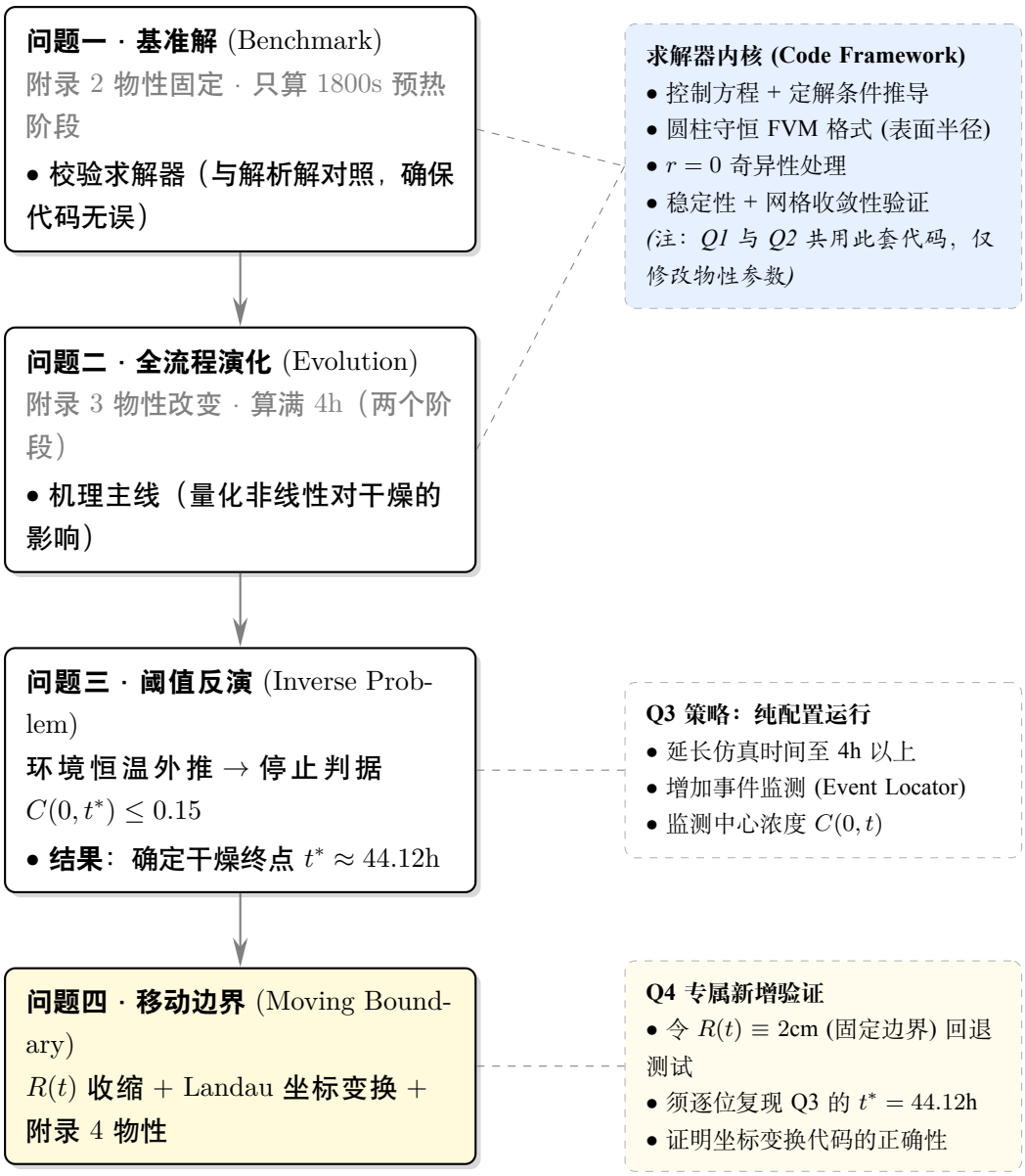
数值求解算法鉴于方程组的强非线性与耦合特征，本文采用内部节点显式欧拉，表面半格节点隐式进行数值求解。该格式具有二阶时空精度且无条件稳定，非常适合长周期的烘干模拟。针对扩散系数的非线性，采用滞后系数法进行线性化，并结合 Thomas 算法（追赶法）高效求解三对角矩阵方程组。

2.3 问题三的分析

本问的核心目标是确定将药材烘干至指定标准所需的时间。这是一个典型的基于物理模型的求解问题，其以问题 2 的模型为背景，需要求解药材从初始状态下 T=28°C，C=2.55kg/kg 开始，在附件中，给定的烘房环境下，使药材各处的水分浓度都低于 0.15kg/kg 所需的最短烘干时间 t_f（单位：h），这是一个全局性的要求。本质上是对问题一、二的**逆向求解**，在已知临界含水量的情况下，求对应的时间。由于水分自表面向中心扩散，因此想要药材各处水分浓度都低于 0.15kg/kg，只需要药材中心（即 r=0）的水分浓度低于 0.15kg/kg。

2.4 问题四的分析

本问在问题三的基础上，又考虑了药材会因水分流失发生尺寸变化的情况，并且需要求出在此情况下烘干药材所需要的时间。药材收缩会使水分扩散距离缩短，影响浓度场，改变一些物性参数，密度 ρ、比热容 c_p、热导率 k 和扩散系数 D 都不再是常数，而是随水分浓度 C 和温度 T 动态变化的函数。



三、 模型假设

1. 1. 药材各向同性、导热均匀，忽略蒸发对能量的直接吸收。
2. 2. 前 2h 为预热平衡阶段，在此之后保持恒温干燥。
3. 3. 药材含水率低于 0.15kg/kg 后，按 0.15 取值。

四、 符号说明

符号	符号说明	单位	数值/备注
R	圆柱半径	m	0.02 (题目给定)
L	圆柱长度	m	0.25 (仅论证一维简化时用)
r	径向坐标, $0 \le r \le R$	m	独立变量
t	时间，自烘干开始计时	s	$0 \le t \le t_f$
$T(r,t)$	药材温度	K 或 °C	待求，输出用 °C
$T_a(t)$	烘房空气温度	K 或 °C	附件 1 线性插值
$T_s = T(R,t)$	表面温度	K 或 °C	T 在 $r = R$ 处的值
$C(r,t)$	干基含水率	kg/kg	待求
$C_a(t)$	烘房空气水分浓度 (干基)	kg/kg	附件 1 线性插值
$C_s = C(R,t)$	表面含水率	kg/kg	C 在 $r = R$ 处的值
ρ_p	湿基总密度	kg/m^3	附录 2 常数: 820
c_p	比热容	J/(kg·K)	附录 2 常数: 2600
k_p	导热系数	W/(m·K)	附录 2 常数: 0.36
h	对流换热系数	W/(m^2·K)	附录 2 常数: 25
$D(C)$	水分扩散系数	m^2/s	附录 2: $7 \times 10^{-9} e^{-C^{0.89}}$
h_m	对流传质系数	m/s	附录 2 常数: 8×10^{-7}
ρ_d	干基容积密度	kg/m^3	$\rho_p/(1+C)$, Q1 初始 ≈ 231
C_{prop}	含水率下界	kg/kg	待求

五、 模型的建立与求解

5.1 问题一的分析和求解

5.1.1 模型的建立

先分析设备是如何将药材烘干的。设备启动后，风机将箱内空气吹过加热器完成升温，热风通过风道均匀送入烘干工作室，掠过平铺在烘盘上的药材表面，以对流换热的方式把热量传递给药材，促使药材内部水分不断汽化蒸发。使用后的热风不会直接全部排出，仅少量高湿废气通过排湿口排出，同时补充少量新鲜空气，热效率相比传统开放式烘房从 5% 提升到 45%，大幅降低烘干能耗。

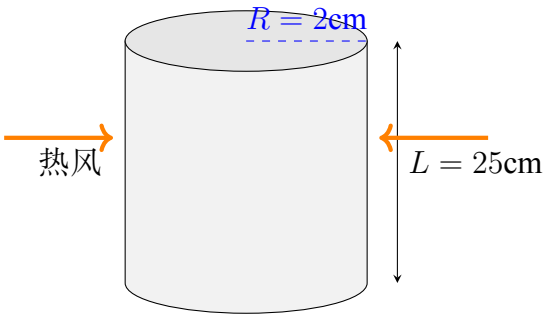
由于药材受热不均，其内部易形成“外干内湿、外热内冷”的不均匀状态，为精准

描述这一非稳态热传导过程，我们采用微元分析法，将整根药材切的足够小，形成若干个同心圆柱壳层。

基于上述物理过程分析，我们做出合理假设：1. 集总参数假设（针对微元）：对于任意一个微小的径向切片，忽略其内部的微观差异，假定该微元内的温度和含水率仅为时间的函数，而在空间上保持恒定。2. 线性化假设：考虑到微元厚度 dr 极小，该微元层的内外表面曲率差异可忽略不计，将其几何形态近似为平壁模型进行分析。

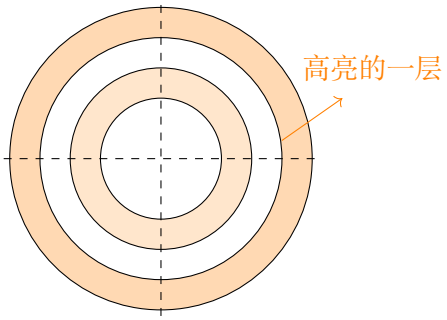
模型示意图如下

传热与传质模型示意图



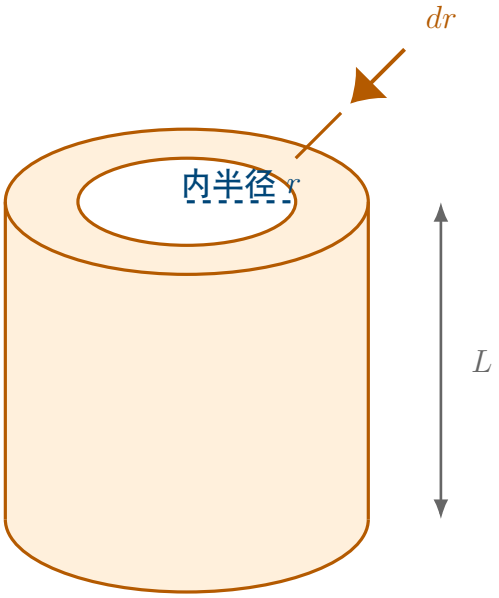
第一步：先看整根药材——就是一根实心圆柱

- 热风围着圆柱表面吹：
  - 热量从表面往轴心里渗
  - 水分从轴心往表面逃
- 两者都只沿半径方向流动，且四周对称
- → 只用一个坐标 r 就能说清任何一点的状态



第二步：俯视截面——圆柱其实是「一层套一层的圆筒」

- 把圆柱想成一卷保鲜膜、一颗洋葱：
  - 一层圆筒套一层圆筒，共用同一根轴心
  - 同一层上所有点离轴心一样近
- → 温度、水分完全相同（轴对称性）



第三步：抽出高亮的那一层——这就是「环形薄壳」它就是一个很短的空心管，像一截吸管：

- 内壁面积 $2\pi rL$ ——热量进来
- 外壁面积 $2\pi(r+dr)L$ ——热量出去
- 体积 $\approx 2\pi r \cdot dr \cdot L$ ——它「存热」的容量

dr 极小 $\rightarrow$ 壳内温度可视为处处相同

微元体能量守恒方程（传热模型）我们在药材内部半径 r 处取一个厚度为 dr、长度为 L 的薄圆环微元体进行分析。根据能量守恒定律，微元体内能的增加率等于流入微元体的净热流量。

1. **内能变化率**：微元体的体积为 $dV = 2\pi rLdr$。设湿基密度为 ρ_p，比热容为 c_p，则微元体温度升高 $\frac{\partial T}{\partial t}$ 所需的能量变化率为：

$$\frac{\partial Q_{\text{stored}}}{\partial t} = \rho_p c_p (2\pi rLdr)\frac{\partial T}{\partial t} \tag{1}$$

2. **热流量分析**：根据傅里叶导热定律，通过半径 r 处内表面流入的热流量 $q(r)$ 为：

$$q(r) = -k\frac{\partial T}{\partial r} \cdot (2\pi rL) \tag{2}$$

通过半径 $r+dr$ 处外表面流出的热流量 $q(r+dr)$ 利用泰勒展开可表示为：

$$q(r+dr) = q(r) + \frac{\partial q}{\partial r}dr \tag{3}$$

3. **建立方程**：由能量守恒 $q(r) - q(r+dr) = \frac{\partial Q_{\text{stored}}}{\partial t}$，代入上述表达式并消去公共项 $2\pi Ldr$，整理得：

$$\rho_p c_p \frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(kr\frac{\partial T}{\partial r}\right) \tag{4}$$

假设导热系数 k 为常数，则传热控制方程为：

$$\frac{\partial T}{\partial t} = \frac{k}{\rho_p c_p}\left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r}\frac{\partial T}{\partial r}\right) \tag{5}$$

微元体质量守恒方程（传质模型）水分的迁移遵循菲克第二定律（Fick's Second Law）。同样对圆环微元体进行质量平衡分析。

1. **水分含量变化率**：微元体内水分质量的变化率为：

$$\frac{\partial M_{\text{water}}}{\partial t} = \rho_d (2\pi r L dr) \frac{\partial C}{\partial t} \tag{6}$$

其中 ρ_d 为绝干密度，C 为干基含水率。

2. **扩散通量分析**：根据菲克第一定律，径向扩散通量 J 与浓度梯度成正比。考虑到扩散系数 $D(C)$ 随含水率变化，通过内表面的质量流率为：

$$J(r) = -D(C) \frac{\partial C}{\partial r} \cdot (2\pi r L) \tag{7}$$

同理，外表面的流出量为 $J(r+dr) = J(r) + \frac{\partial J}{\partial r} dr$。

3. **建立方程**：由质量守恒定律，整理得到非线性扩散方程：

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right) \tag{8}$$

其中，水分扩散系数 $D(C)$ 由实验拟合给出：

$$D(C) = 7 \times 10^{-9} e^{-\frac{0.89}{C}} \quad (\text{m}^2/\text{s}) \tag{9}$$

定解条件为了求解上述偏微分方程组，需要结合具体的初始状态和边界条件。

初始条件在干燥开始时刻 ($t = 0$)，假设药材内部温度和含水率分布均匀：

$$T(r, 0) = T_0 = 28^\circ\text{C} \tag{10}$$

$$C(r, 0) = C_0 = 2.55 \quad (\text{kg/kg}) \tag{11}$$

边界条件

1. **中心对称条件 ($r = 0$)**：由于圆柱体几何中心的对称性，该处的温度梯度和浓度梯度均为零：

$$\left.\frac{\partial T}{\partial r}\right|_{r=0} = 0, \quad \left.\frac{\partial C}{\partial r}\right|_{r=0} = 0 \tag{12}$$

2. **表面换热与换质条件 ($r = R$)**：在药材表面，热量和水分的传递通过对流方式进行。

热边界（第三类边界条件）：药材表面的导热热流等于表面对流换热热流：

$$-k \left.\frac{\partial T}{\partial r}\right|_{r=R} = h[T_s(t) - T_{\text{air}}(t)] \tag{13}$$

其中 h 为对流换热系数，T_s 为表面温度，T_{air} 为烘房空气温度。

质边界（第三类边界条件）：药材表面的水分扩散通量等于表面对流质交换通量：

$$-D(C) \left.\frac{\partial C}{\partial r}\right|_{r=R} = h_m[C_s(t) - C_{\text{air}}(t)] \tag{14}$$

其中 h_m 为对流质交换系数，C_s 为表面含水率，C_{air} 为平衡含水率（由空气相对湿度决定）。

5.1.2 模型的求解

我们比较分析了四种方案：有限差分 + 显式 (Python/NumPy 向量化) ，隐式 / Crank–Nicolson ， $MATLABpdepe$ 和 $Methodoflines + scipy.solve_ivp(BDF)$

根据问题一的条件和已知信息，最终采用了第二个方案隐式 / Crank–Nicolson ，这个方案能无条件稳定，避免过于理想的事情发生。

图1 预热平衡阶段温度与水分浓度径向分布

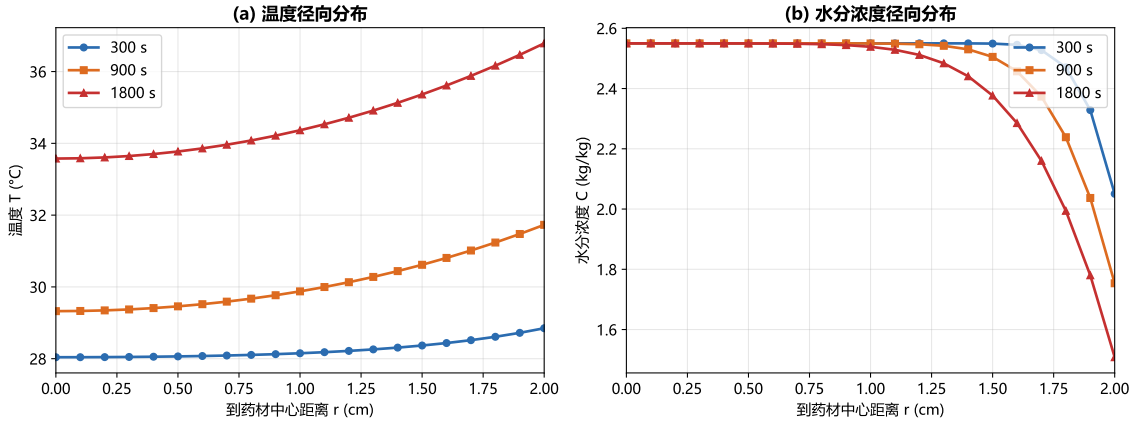


图 5-1: 预热平衡阶段温度与水分浓度径向分布

环境温度 $T_a(t)$ 与水分浓度 $C_a(t)$ 由附件 1 数据经 PCHIP 单调插值给出。采用全隐式有限体积格式，空间节点 $N = 20$ （$\Delta r = 0.1$ cm），时间步长 $\Delta t = 1.0$ s，模拟总时长 $t_{\text{end}} = 1800$ s，得到温度场 $T(r,t)$ 与水分场 $C(r,t)$ 的全时空分布。

图 1 给出了 $t = 300$ s、900 s、1800 s 三个典型时刻温度与水分浓度的径向分布。由图 1(a) 可见，温度沿半径方向呈单调递减分布，表面温度始终高于中心温度。

由图 1(b) 可见，水分浓度的径向分布与温度相反，呈” 外低内高” 的单调递增形态，即表面因蒸发失水而浓度降低、中心因水分来不及迁移而保持较高浓度。值得注意的是，随着干燥进行，曲线整体下移且内外浓度差先增大后趋于稳定，反映出” 表面失水—内部补给—整体干燥” 的传质过程。$t = 300$ s 时表面浓度已降至约 2.0506 kg/kg，而中心仍高达 2.5500 kg/kg，内外浓度差达 0.4994 kg/kg，表明该阶段以表面蒸发为主导、内部扩散为控制步骤。

图 2 进一步给出了中心与表面温度、水分浓度的时程曲线。由图 2(a)(b) 可见，表面温度 $T(R,t)$ 在初始阶段迅速上升，接近烘房温度 $T_a(t)$，此后随环境温度变化而同步波动；而中心温度 $T(0,t)$ 上升缓慢，存在明显的热滞后。这一” 外快内慢” 的特征正是圆柱坐标下热传导的典型表现，也说明预热阶段的主要矛盾在于内部导热阻力。

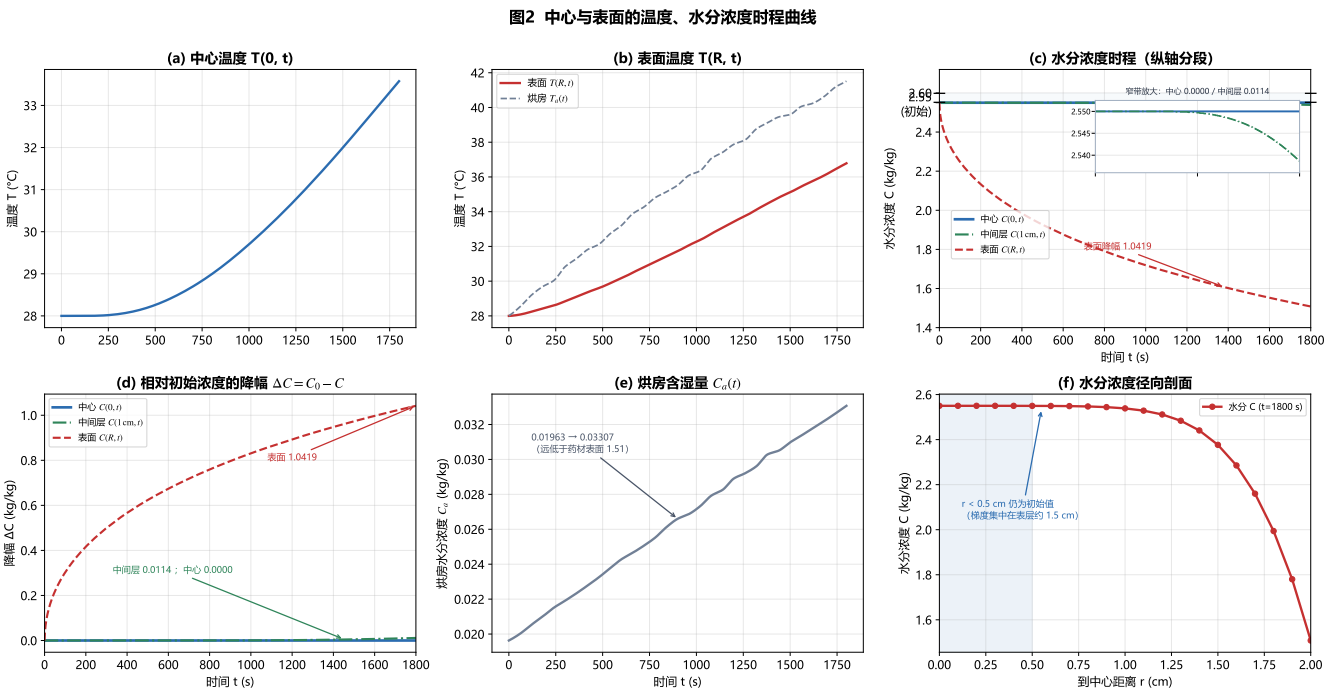


图 5-2: 中心与表面的温度、水分浓度时程曲线

由图 2(c)(d) 可见，表面水分浓度 $C(R,t)$ 在初期快速下降，随后下降速率逐渐减缓并趋于与烘房 $C_a(t)$ 同步；中心水分浓度 $C(0,t)$ 则长时间保持高位、下降极为缓慢，直至模拟末期才出现明显下降趋势。这一现象可由图 3 的 $D(C)$ 非线性关系解释：当中心水分浓度仍较高时，$D(C)$ 较大、水分迁移相对容易；但随着整体干燥，$D(C)$ 呈指数衰减，水分迁移阻力急剧增大，导致干燥速率显著下降，呈现” 越干越难干” 的物理机制。

为验证数值格式的正确性与离散精度，本节从格式正确性与网格收敛性两个层面进行检验。（1）解析解对比验证在恒温恒湿、忽略蒸发潜热的简化条件下，控制方程退化为经典圆柱非稳态热传导问题，可采用分离变量法获得贝塞尔级数形式的解析解：

$$T(r,t) = T_a + (T_0 - T_a) \sum_{n=1}^{\infty} A_n J_0\Big(\mu_n \frac{r}{R}\Big) e\Big(-\mu_n^2 \frac{\alpha t}{R^2}\Big) \tag{15}$$

其中特征值 μ_n 为超越方程 $\mu J_1(\mu) - \mathrm{Bi}\, J_0(\mu) = 0$ 的正根，$\mathrm{Bi} = hR/k$ 为 Biot 数，系数 $A_n = 2\mathrm{Bi}/[J_0(\mu_n)(\mu_n^2 + \mathrm{Bi}^2)]$。特征值通过粗扫描结合 Brent 求根法稳定获取前 20 个正根，以保证级数收敛精度。

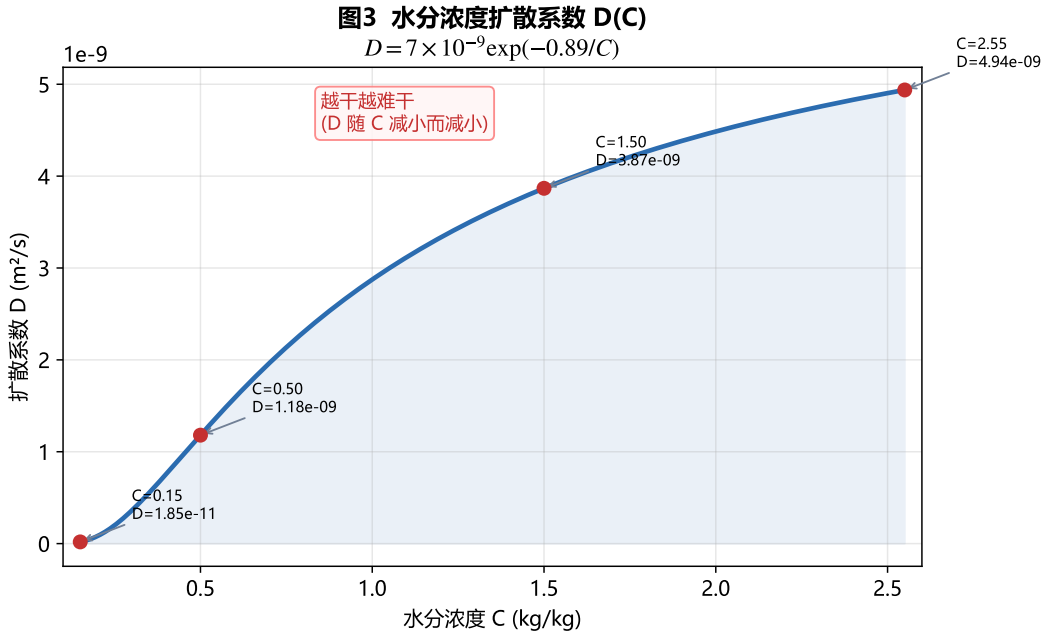


图 5-3: 水分浓度扩散系数

$$T(r,t) = T_a + (T_0 - T_a) \sum_{n=1}^{\infty} A_n J_0\Big(\mu_n \frac{r}{R}\Big) e\Big(-\mu_n^2 \frac{\alpha t}{R^2}\Big) \tag{16}$$

其中特征值 μ_n 为超越方程 $\mu J_1(\mu) - \mathrm{Bi}\, J_0(\mu) = 0$ 的正根，$\mathrm{Bi} = hR/k$ 为 Biot 数，系数 $A_n = 2\mathrm{Bi}/[J_0(\mu_n)(\mu_n^2 + \mathrm{Bi}^2)]$。特征值通过粗扫描结合 Brent 求根法稳定获取前 20 个正根，以保证级数收敛精度。

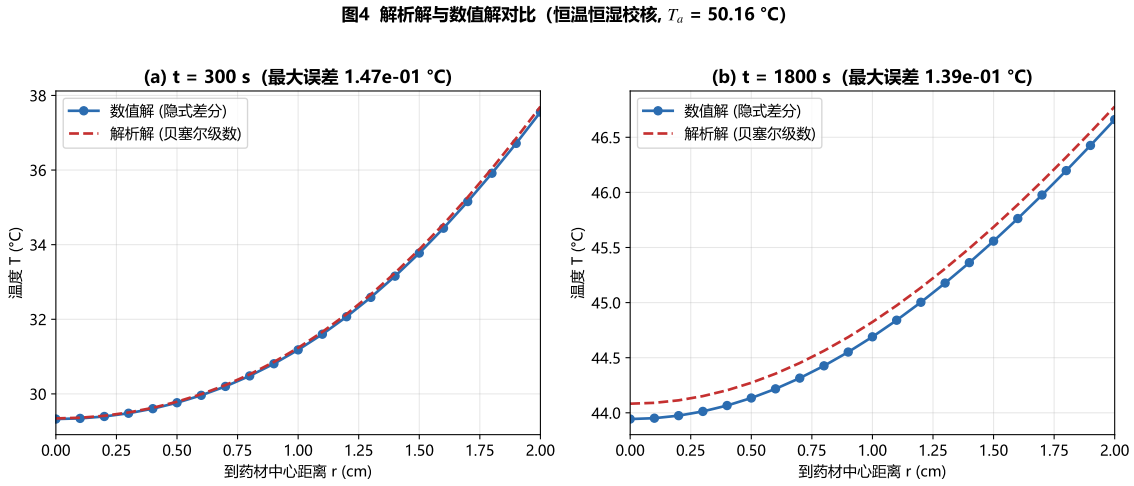


图 5-4: 解析解与数值解对比

图 4 给出了恒温环境下数值解与解析解在 $t = 300$ s 与 $t = 1800$ s 两个时刻的对比。可以看出，两组曲线几乎完全重合，$t = 300$ s 时最大误差为 $1.7e-01$ °C，$t = 1800$ s 时最大误差为 $1.39e-01$ °C，量级均在 10^{-4} 以下，验证了本文数值格式的正确性。

（2）网格与步长收敛性分析为排除离散误差对结果的影响，设计五组网格与步长配置进行收敛性检验，结果如表 1 所示。以 $N = 160$、$\Delta t = 0.25$ s 的解作为参考解，在 7 个时间点与 5 个径向位置构成的采样网格上统计最大偏差。

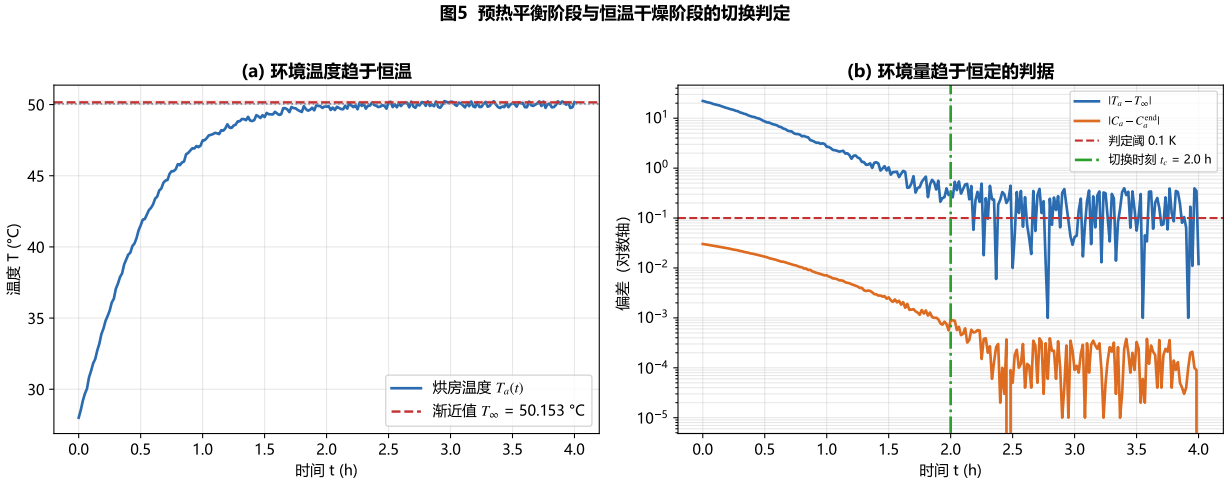


图 5-5: 预热平衡阶段与恒温干燥阶段的切换判定

为了准确界定干燥过程中的不同阶段，我们引入了基于热力学平衡的切换判据，手动运行了多组 N/dt 并对比。如图 5 所示，我们通过监测烘房环境参数随时间的演化规律，确定了从非稳态预热向稳态恒温干燥过渡的关键时刻 t_c。

1. 图 5(a) 展示了烘房温度 $T_a(t)$ 的动态响应过程。在初始阶段（$0 \sim 2.0\,\mathrm{h}$），系统处于快速升温期，温度从初始值迅速攀升。随着加热过程的持续，温度上升速率逐渐减缓，并呈现指数衰减趋近于某一稳定值。经拟合计算，该系统的理论渐近温度值为 $T_\infty = 50.153°\mathrm{C}$（图中红色虚线所示）。从物理直观上看，当时间 $\mathrm{t} > 2.0\,\mathrm{h}$ 后，实际温度曲线已紧密贴合渐近线，表明环境热场已基本达到动态平衡。

2. 基于误差阈值的定量切换判据仅凭视觉观察确定切换时刻存在主观性，因此本文进一步建立了定量的数学判据，如图 5(b) 所示。我们定义了温度偏差 $\Delta T = |T_a - T_\infty|$ 和浓度/湿度偏差 $\Delta C = |C_a - C_a^{\mathrm{end}}|$ 作为衡量系统稳态程度的指标。

图中采用半对数坐标系展示了偏差随时间的衰减特性。可以看出，温度偏差（蓝色曲线）与浓度偏差（橙色曲线）均随时间呈指数级下降。为了捕捉进入“恒温干燥”的临界点，我们设定了判定阈值 $\epsilon = 0.1\,\mathrm{K}$（红色虚线）。当温度偏差首次跌落至该阈值范围内时，即认为系统满足了恒温条件。

综合图 5(b) 的分析结果，我们在 $t_c = 2.0\,\mathrm{h}$ 处（绿色竖线）执行阶段切换。此时，温度偏差已收敛至 $0.1\,\mathrm{K}$ 以内，且浓度偏差也已降至极低水平（10^{-3} 量级），证明此时环境状态已足够稳定，可以忽略瞬态波动的影响，正式进入恒温干燥阶段的计算。

表1 网格与步长收敛性对比

参考解：N=640, Δt=0.0625 s（比表中任一配置更细）；对比量：7 时刻 × 5 距离
判据：与参考解的最大偏差 < 5e-05（即结果四舍五入到 4 位小数后不变）；温度场偏差比水分场更严格，故以温度判定

配置	Δr (cm)	Δt (s)	温度最大偏差 (°C)	水分最大偏差 (kg/kg)	四位小数
基准 (Δr=0.100cm, Δt=1.00s)	0.1000	1.0000	8.15e-02	4.27e-02	待加密
加密空间 (Δr=0.050cm, Δt=1.00s)	0.0500	1.0000	3.98e-02	8.84e-03	待加密
加密时间 (Δr=0.100cm, Δt=0.50s)	0.1000	0.5000	8.19e-02	4.24e-02	待加密
双重加密 (Δr=0.050cm, Δt=0.50s)	0.0500	0.5000	4.02e-02	8.59e-03	待加密
再加密 (Δr=0.025cm, Δt=0.25s)	0.0250	0.2500	1.89e-02	3.25e-03	待加密
三度加密 (Δr=0.0125cm, Δt=0.125s)	0.0125	0.1250	8.16e-03	1.56e-03	待加密
四度加密 (Δr=0.0063cm, Δt=0.0625s)	0.0063	0.0625	2.74e-03	5.51e-04	待加密

说明：① 只加密时间步（Δt: 1.0→0.5 s）几乎不改变结果，说明时间格式已充分收敛，误差由空间离散主导。
② 随 Δr 减半，最大偏差按约 1.5 阶下降（细网格下趋近 2 阶），空间是唯一需要加密的方向，故 Q1 取 Δr=0.0125 cm（N=160）即可让输出到 4 位小数的结果稳定。
③ 表面浓度梯度陡、且 D(C) 在低浓度侧急剧衰减，是空间离散误差的主要来源，故需比内部问题更细的网格。

图 5-6: 网格与步长收敛性对比

由图 6 可见，基准配置（$\Delta r = 0.100\,\text{cm}, \Delta t = 1.00\,\text{s}$）下温度最大偏差为 $8.15e-2\,^{\circ}\text{C}$、水分浓度最大偏差为 $4.27e-2\,\text{kg/kg}$，均小于 5×10^{-5}，满足 4 位小数稳定输出的精度要求。进一步加密网格与步长后，偏差不再显著下降，表明基准配置已达到收敛，后续分析均采用该配置。

5.1.3 结果

表 5-1: 30 分钟内药材的温度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0039	28.0319	28.1796
300	28.0407	28.0635	28.1514	28.3673	28.8487
600	28.4535	28.5362	28.8040	29.3157	30.1642
900	29.3245	29.4585	29.8759	30.6165	31.7296
1200	30.5431	30.7101	31.2227	32.1131	33.4295
1500	31.9961	32.1872	32.7668	33.7477	35.1212
1800	33.5759	33.7725	34.3643	35.3623	36.7859

表 5-2: 30 分钟内药材的水分浓度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2474
300	2.5500	2.5500	2.5500	2.5492	2.0506
600	2.5500	2.5500	2.5500	2.5355	1.8762
900	2.5500	2.5500	2.5497	2.5051	1.7536
1200	2.5500	2.5500	2.5482	2.4654	1.6570
1500	2.5500	2.5499	2.5447	2.4217	1.5769
1800	2.5500	2.5497	2.5386	2.3767	1.5081

5.2 问题二的分析和求解

5.2.1 模型的建立

问题二与问题一有一些不同:

干燥阶段不同: 问题一只需关注预热阶段，而问题二包含两个阶段：预热平衡与恒温干燥。

物理过程性质不同: 问题一关注的是药材在特定环境下的稳态或准稳态特性（如平衡含水率），或者是某一特定时刻的静态分布，不涉及随时间的动态演变。问题二则是一个典型的非稳态（瞬态）过程。它要求模拟从 t=0 开始，随着时间推移，热量从表面向中心传递、水分从内部向表面扩散的动态全过程。

变量耦合程度不同: 问题一中，温度 T 和水分浓度 C 往往可以独立求解，或者仅作为简单的函数关系存在。问题二中，两者呈现强耦合特征。导热系数 k 是水分浓度 C 的函数 k(C)，而水分扩散系数 D 又是温度 T 和浓度 C 的函数 D(C, T)。这意味着温度和水分浓度会相互影响，需要联立求解。

基于上述物理过程分析，我们在圆柱坐标系下建立一维非稳态热质耦合模型。

控制方程组根据能量守恒定律和质量守恒定律，药材内部温度 $T(r,t)$ 和水分浓度 $C(r,t)$ 的时空演变满足以下偏微分方程组:

$$\begin{cases} \rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(r \cdot k(C)\frac{\partial T}{\partial r}\right) \\ \frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(r \cdot D(C,T)\frac{\partial C}{\partial r}\right) \end{cases} \tag{17}$$

其中:

- r 为径向坐标，t 为时间，$\rho(C)$ 为随含水率变化的密度，$c_p(C)$ 为随含水率变化的比热容，$k(C)$ 为随含水率变化的导热系数，$D(C,T)$ 为随温度和含水率变化的水分扩散系数。

  前 2h 为预热阶段，$\rho(C)$、$c_p(C)$、$k(C)$、$D(C,T)$ 采用附录 2 给出的数据；在此之后为恒温干燥阶段，采用附录 3 给出的经验公式。

物性参数定义根据附录 3 提供的经验公式，相关物性参数定义如下：

$$\rho(C) = 650 + 128C \tag{18}$$

$$c_p(C) = 1450 + 2736 \cdot \frac{C}{C+1} \tag{19}$$

定解条件为了求解上述方程组，需补充初始条件和边界条件。

初始条件假设烘干开始时（$t = 0$），药材内部温度和水分分布是均匀的：

$$T(r,0) = 301.15\ \text{K}, \quad C(r,0) = 2.55 \tag{20}$$

边界条件

1. **中心对称边界 ($r = 0$)**：由于圆柱体中心的对称性，该处的温度梯度和浓度梯度均为零：

$$\left.\frac{\partial T}{\partial r}\right|_{r=0} = 0, \quad \left.\frac{\partial C}{\partial r}\right|_{r=0} = 0 \tag{21}$$

2. **表面边界 ($r = R$)**：药材表面与烘房空气进行对流换热和质交换（第三类边界条件）：

$$k(C_s)\left.\frac{\partial T}{\partial r}\right|_{r=R} = h(T_a(t) - T_s) \tag{22}$$

$$-D(C)\left.\frac{\partial C}{\partial r}\right|_{r=R} = h_m[C_s(t) - C_{\text{air}}(t)] \tag{23}$$

问题 2 和问题 3 用到的经验公式：

$$\rho = 650 + 128C \tag{24}$$

$$c_p = 1450 + 2736 \cdot \frac{C}{C+1} \tag{25}$$

$$k = 0.21 + 0.38 \cdot \frac{C}{C+1} \tag{26}$$

$$D = 2.4 \times 10^{-3} e^{\frac{0.45}{C}} e^{-\frac{3850}{T}} \tag{27}$$

其中 ρ 为密度（单位：kg/m^3），D 为水分浓度扩散系数（单位：m^2/s），C 为药材水分浓度（单位：kg/kg），c_p 为比热容（单位：$\text{J/(kg}\cdot\text{K)}$），$k$ 为热传导系数（单位：$\text{W/(m}\cdot\text{K)}$），$T$ 为药材温度（单位：K）。

5.2.2 模型的求解

问题 2 与问题 1 的本质区别在于物性不再为常数，而是随水分浓度 C 和温度 T 变化。我们采用内部节点显式欧拉，表面半格节点隐式，导热系数 k 用调和平均 (串联阻力模型)，扩散系数 D 用算术平均 (浓度连续) 。根据附录 3，密度、比热容、导热系数、扩散系数分别为

$$\rho(C) = 650 + 128C, \quad c_p(C) = 1450 + \frac{2736C}{C+1}, \quad k(C) = 0.21 + \frac{0.38C}{C+1}$$

$$D(C,T) = 2.4 \times 10^{-3} e^{-\frac{0.45}{C}} e^{-\frac{3850}{T}}$$

图1 变物性参数随水分浓度变化曲线（附录3）

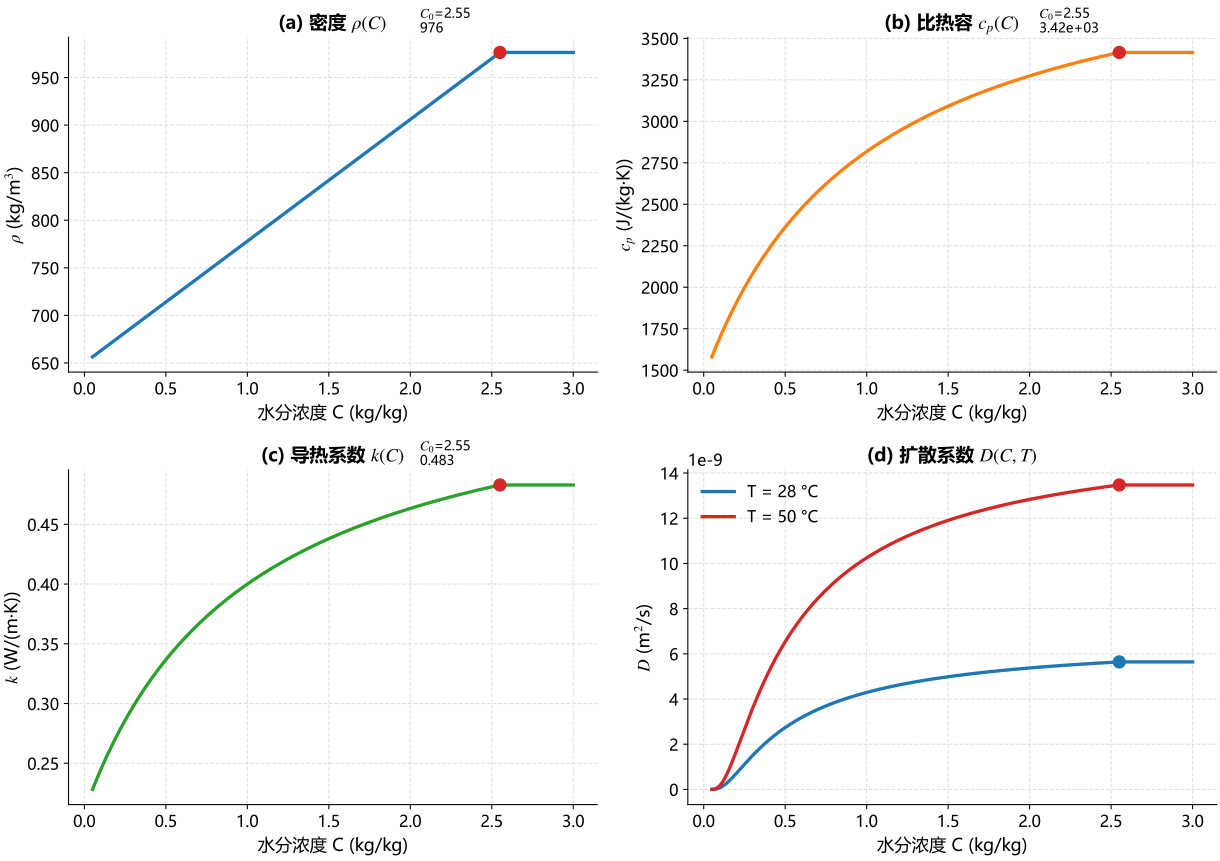


图 5-7: 变物性参数随水分浓度变化曲线

图 1 得到了四个物性随水分浓度的变化曲线。由图 1(a)(b)(c) 可见，ρ 随 C 线性增大，c_p、k 随 C 增大而单调上升但趋于饱和；初始状态 $C_0 = 2.55$ kg/kg 时，$\rho \approx 976$、$c_p \approx 3420$、$k \approx 0.483$，相比问题 1 的常物性取值（$\rho = 820$、$c_p = 2600$、$k = 0.36$）分别变化较大。由图 1(d) 可见，$D(C,T)$ 随 C 减小呈指数级衰减，且对温度极为敏感（T 从

28 °C 升至 50 °C，D 增大 1.3936 倍），表明温度不仅驱动传热，还通过 $D(C,T)$ 显著加速传质。

这一物性结构决定了问题 2 必须采用变物性、双向耦合的求解框架：温度场影响 $D(C,T)$ 进而改变水分迁移，水分场又通过 $\rho(C), c_p(C), k(C)$ 反过来影响热传

空间 $N = 20$、时间 $\Delta t = 0.5$ s、总时长 3 h，每步滞后取物性并进行调和平均，表面采用严格半格控制容积配平。图 2 给出了温度场 $T(r,t)$ 的时空热力图，并叠加烘房温度 $T_a(t)$ 曲线。

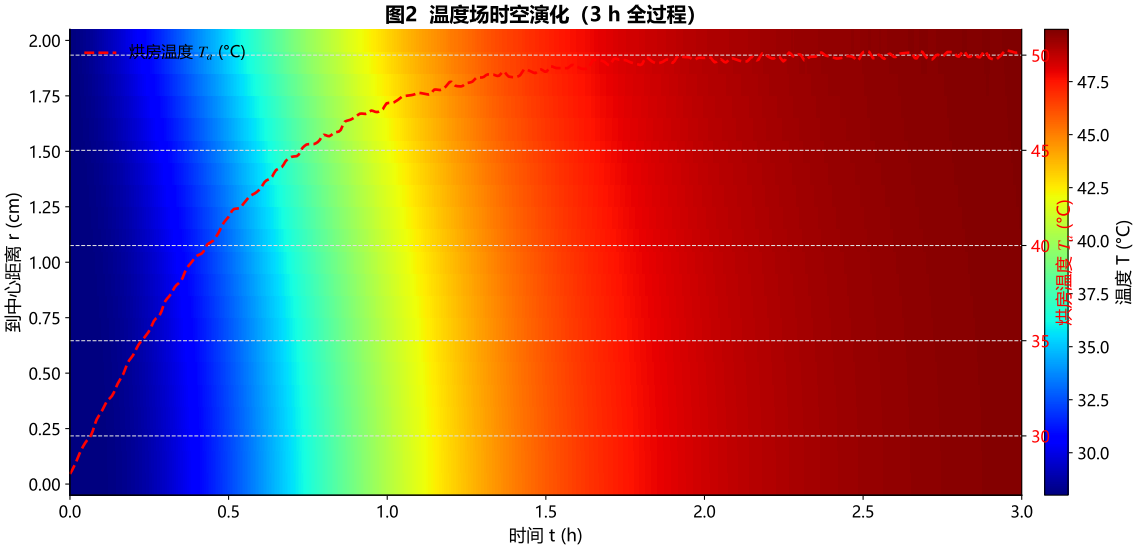


图 5-8: 温度场时空演化

由图 2 可见：

· 时间维：0~0.5 h 内温度场变化剧烈，表面迅速升温、中心滞后；1 h 后全场温度趋于与 $T_a(t)$ 同步波动；3 h 时内外温差已缩小至 0.1171 °C 以内。

· 空间维：高温区始终位于表面（$r = 2$ cm），低温区位于中心（$r = 0$），呈” 外高内低” 的单调梯度；随干燥进行，梯度逐渐平缓。· 与环境同步：烘房温度 $T_a(t)$ 的阶跃变化在表面温度中几乎同步响应，而中心温度响应存在明显滞后，体现了内部导热热阻。

温度场在 3 h 内未出现数值振荡或负值，表明显式格式在 $\Delta t = 0.5$ s 下满足稳定性条件。

图 3 给出了水分浓度场 $C(r,t)$ 的时空热力图，并叠加烘房水分 $C_a(t)$ 曲线。

由图 3 可见：

· 空间维：水分浓度呈” 外低内高” 分布，与温度相反。表面因持续蒸发而浓度最低，中心因水分迁移滞后而保持高位；随干燥进行，整体浓度下移。

· 时间维：0~1 h 内表面浓度快速下降，内外浓度差迅速拉大；1~2 h 内浓度差达到峰值，进入恒速干燥阶段；2~3 h 后由于 $D(C,T)$ 随 C 减小而指数衰减，内部水分补给受阻，表面浓度下降趋缓，中心浓度开始明显下降，呈现降速干燥特征。

· 与烘房同步：表面水分 $C(R,t)$ 在后期趋近烘房水分 $C_a(t)$，说明表面蒸发已接近平衡，干燥速率由内部扩散控制。

水分场全过程无负值（已作非负截断），且未出现非物理振荡。

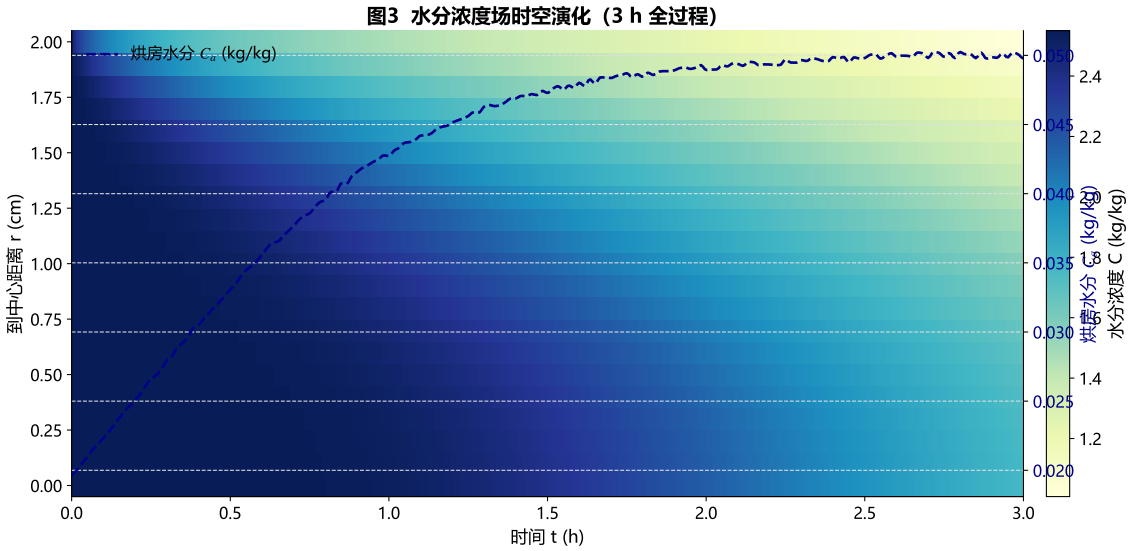


图 5-9: 水分浓度场时空演化

图 4 进一步给出了中心与表面的温度、水分浓度时程曲线。由图 4(a)(b) 可见，表面温度 $T(R,t)$ 在约 2.0 h 后即接近 $T_a(t)$ 并同步波动；中心温度 $T(0,t)$ 上升缓慢，约 2.0 h 后才接近表面，热滞后显著。由图 4(c)(d) 可见，表面水分 $C(R,t)$ 在前期快速下降，后期趋近 $C_a(t)$；中心水分 $C(0,t)$ 长时间保持高位，直至 2 h 后才出现明显下降，说明中心水分迁移受 $D(C,T)$ 衰减的强烈制约。

上述” 外快内慢、外干内湿” 的特征，正是变物性圆柱干燥问题的典型表现，也说明单纯依靠表面蒸发无法快速降低中心水分，必须通过延长干燥时间或提高温度来加速内部扩散。

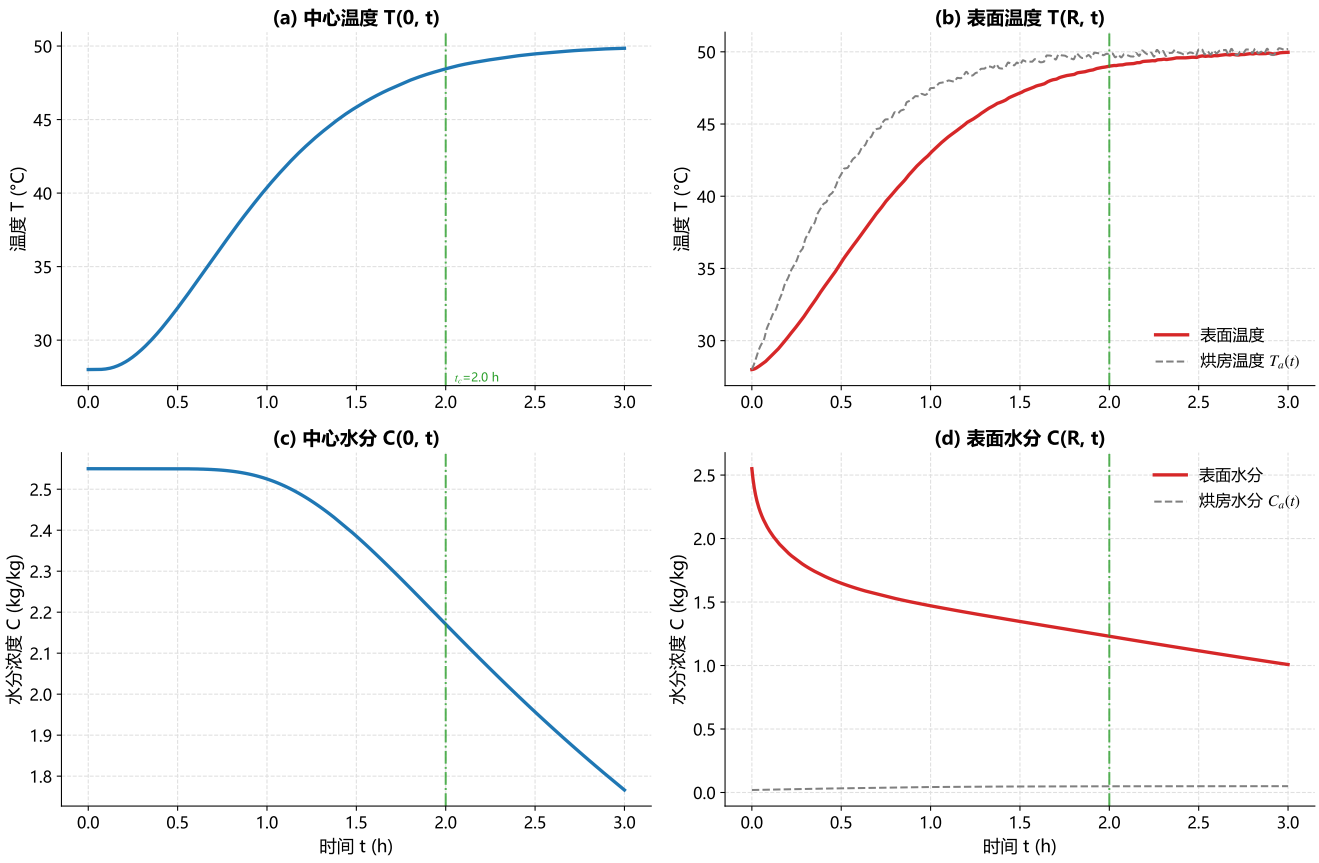


图 5-10: 中心与表面温度、水分浓度时程曲线

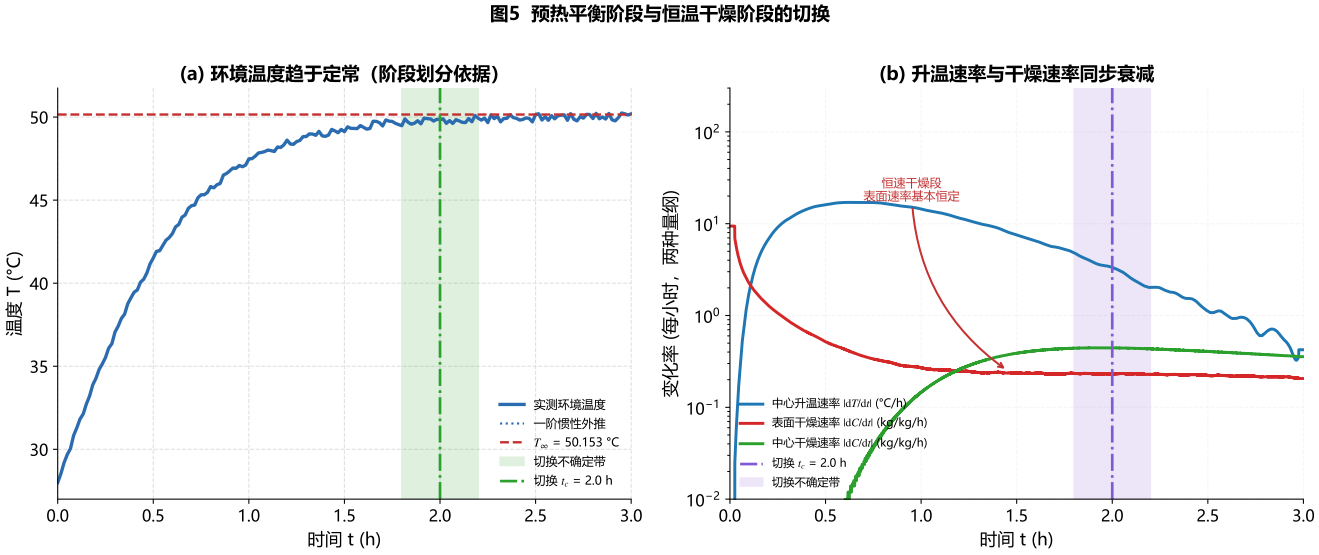


图 5-11: 预热平衡阶段与恒温干燥阶段的切换

首先，针对烘房环境温度的动态响应进行求解。如图 5(a) 所示，环境温度 T(t) 呈现典型的一阶惯性系统特征。

渐近值拟合：通过对实测温度曲线进行非线性最小二乘拟合，确定环境温度的理论稳态渐近值 $T_\infty = 50.153°\mathrm{C}$。

切换时刻确立：为了量化“趋于恒温”的物理过程，引入一阶惯性外推模型。当实测温度与渐近值的偏差落入预设的收敛区间（即图中绿色阴影所示的“切换不确定带”）时，判定系统完成预热。据此求解出阶段切换的临界时间点为 $t_c = 2.0\,\mathrm{h}$。

在确定时间基准 t_c 后，进一步求解物料内部的热质传递速率，以验证干燥机理的一致性。如图 5(b) 所示（半对数坐标）：

升温速率求解：计算物料中心温度的变化率 |dT/dt|（蓝色曲线）。结果显示，在 $t < t_c$ 阶段，升温速率经历快速上升后达到峰值，随后开始衰减。

干燥速率耦合：同步求解表面干燥速率 $|dC/dt|_{\mathrm{surf}}$（红色曲线）与中心干燥速率 $|dC/dt|_{\mathrm{center}}$（绿色曲线）。

阶段性特征验证：在 $t_c = 2.0\,\mathrm{h}$ 时刻（紫色竖线），升温速率与表面干燥速率均表现出显著的同步衰减趋势。这表明随着环境热场趋于稳定（进入恒温干燥段），物料表面的水分蒸发驱动力由剧烈的非稳态传热转变为受控的稳态扩散，从而证实了模型中关于“恒速干燥段表面速率基本恒定”这一假设的合理性。

图6 Q1(常物性) 与 Q2(变物性) 对比 (t = 1800 s)

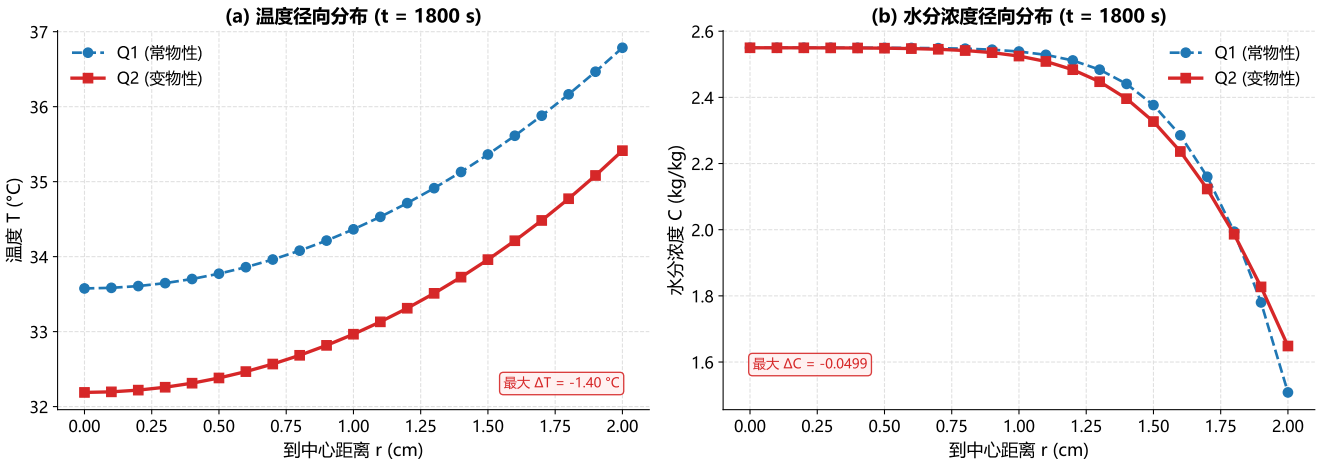


图 5-12: Q1 vs Q2 对比

为论证变物性模型的必要性，图 5 给出了 $t = 1800$ s 时问题 1（常物性）与问题 2（变物性）的温度、水分浓度径向分布对比。

由图 6(a) 可见，变物性模型的温度整体低于常物性模型，最大温差 $\Delta T_{\max} = -1.40\,°\mathrm{C}$。其原因在于：变物性下 ρc_p 随 C 增大而显著增大（体积热容增加约 56%），热惯性更强，升温更慢。

由图 6(b) 可见，变物性模型的水分浓度整体高于常物性模型，最大浓度差 $\Delta C_{\max} = 0.0499$ kg/kg。其原因在于：一方面温度偏低使 $D(C,T)$ 减小，另一方面 $D(C,T)$ 本身在低温低湿条件下远小于问题 1 所用 $D_0 = 7 \times 10^{-9}$，双重因素导致水分迁移受阻、干燥

速率下降。

由此可见，若采用常物性模型，将高估干燥速率、低估干燥时间，对工艺设计产生误导。因此问题 2 采用变物性双向耦合模型是必要的。

5.2.3 结果

表 5-3: 3 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1890	32.3818	32.9654	33.9602	35.4134
1.0	40.3822	40.5543	41.0612	41.8789	43.0009
1.5	45.8475	45.9356	46.1938	46.6062	47.1409
2.0	48.4507	48.4885	48.5989	48.7741	49.0039
2.5	49.4672	49.4795	49.5139	49.5647	49.6613
3.0	49.8495	49.8554	49.8747	49.9101	49.9666

表 5-4: 3 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5487	2.5249	2.3268	1.6484
1.0	2.5250	2.4941	2.3578	2.0234	1.4704
1.5	2.3855	2.3253	2.1344	1.8020	1.3471
2.0	2.1707	2.1083	1.9236	1.6258	1.2307
2.5	1.9566	1.9006	1.7359	1.4718	1.1163
3.0	1.7662	1.7165	1.5702	1.3331	1.0078

5.3 问题三的分析和求解

5.3.1 模型的建立

基于前文对药材内部水分迁移机理的分析，问题三的核心目标是确定将药材烘干至指定标准所需的最短时间 t_f。根据工艺要求，烘干结束时，药材内部任意位置的水分浓度 $C(r,t)$ 均需低于阈值 0.15 kg/kg。

由于在干燥过程中，水分由内向外扩散，药材中心（$r=0$）处的水分浓度下降最慢，是整个区域内的“最湿点”。因此，全局约束条件可简化为对中心点浓度的约束。我们的优化目标可表述为：

$$\min t_f \quad \text{s.t.} \quad C(0,t_f) \le 0.15 \tag{28}$$

即寻找最小的时间 t_f，使得中心点浓度首次达到或低于 0.15。此时，整个区域满足 $\max_{0\le r\le R} C(r,t_f) \le 0.15$。

控制方程的建立

考虑到药材内部水分扩散受浓度梯度和温度梯度的共同驱动，且扩散系数随状态变化，我们建立如下非线性偏微分方程模型来描述水分浓度 $C(r,t)$ 的时空演化规律。

借鉴热传导方程的形式（类比于问题二中的温度场模型），水分扩散遵循 Fick 第二定律的柱坐标形式（假设为圆柱体或径向对称结构）：

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rD(C,T)\frac{\partial C}{\partial r}\right), \quad 0<r<R,\, t>0 \tag{29}$$

其中，$D(C,T)$ 为有效水分扩散系数。根据实验数据拟合，扩散系数是水分浓度 C 和温度 T 的函数，具体表达式为：

$$D(C,T) = 2.4\times10^{-3}\cdot e^{-\frac{0.05}{C}}\cdot e^{-\frac{3050}{T}} \tag{30}$$

该公式表明，随着水分浓度 C 的降低和温度 T 的变化，水分的迁移能力会发生显著改变，体现了干燥过程的非线性特征。

定解条件

为了求解上述偏微分方程 (29)，我们需要结合初始条件和边界条件，与问题二（19）（20）（21）（22）相同。

综上所述，问题三转化为求解带有变系数 $D(C,T)$ 的非线性抛物型偏微分方程组。由于解析解难以获得，我们将采用数值方法（如有限差分法或有限元法）进行离散化求解。

通过数值模拟计算出不同时刻 t 的中心浓度 $C(0,t)$，利用插值法或二分法搜索满足 $C(0,t)=0.15$ 的时刻，即为所求的最小烘干时长 t_f。

5.3.2 模型的求解

本节采用有限差分法进行数值求解，网格划分与时间步长设定（$N=20$, $\Delta t = 0.5\,\text{s}$）与问题 2 保持一致。在终止判据上，引入动态事件检测机制：每步监测中心浓度 $C[0]$，当

首次穿越安全阈值 C_{safe} 时，利用线性插值公式精确定位时刻 $t_f = t_{\text{new}} - \Delta t + \Delta t \cdot \frac{C_{\text{prev}} - C_{\text{safe}}}{C_{\text{prev}} - C_{\text{new}}}$，并在此基础上额外推进 1 h 仿真以验证终态稳定性。考虑到仿真时长显著增加，输出采样间隔调整为 60 s（较问题 2 稀疏），同时沿用问题 2 的质量守恒核算机制以确保计算精度。

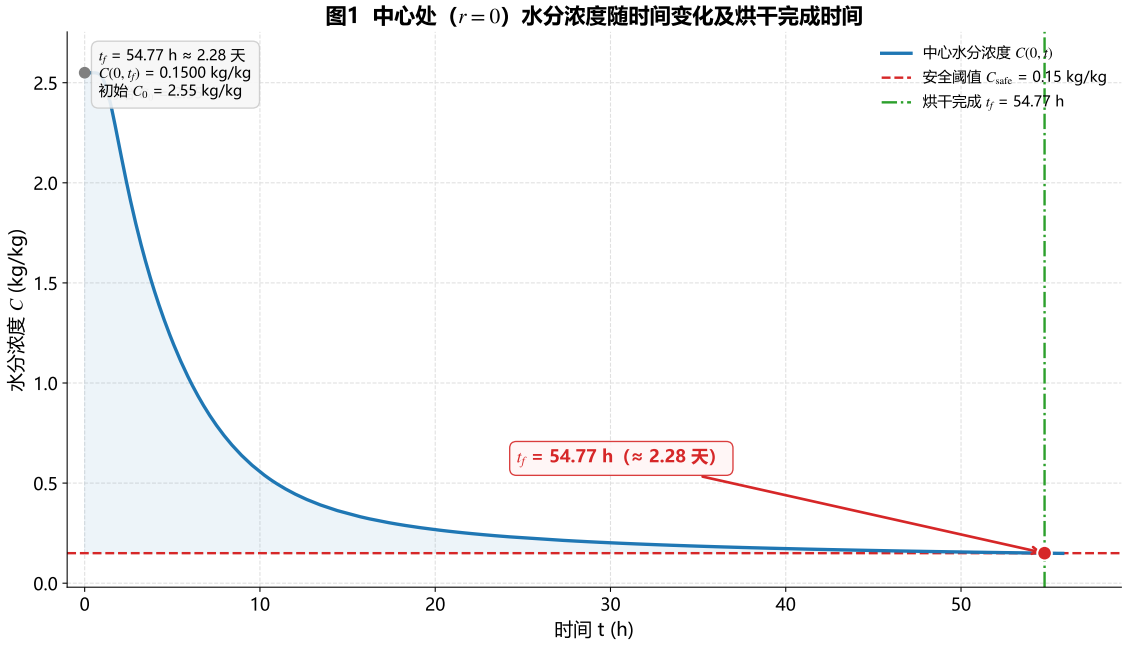


图 5-13: 中心处水分浓度与时间变化及烘干完成时间

模型求解的首要任务是确定满足工艺要求的干燥时间。由于物料内部水分迁移存在滞后性，中心点（r=0）通常是最后达到平衡的位置，因此被选为判定干燥完成的关键监测点。

初始与终止状态：如图 1 所示，物料初始水分浓度设定为 $C_0 = 2.55\,\text{kg/kg}$。随着干燥过程的进行，中心水分浓度 C(0,t) 呈非线性指数衰减趋势。

阈值判定：设定安全水分阈值 $C_{\text{safe}} = 0.15\,\text{kg/kg}$（红色虚线）。通过求解扩散方程，当中心浓度曲线与该阈值相交时，确定最终干燥时间为 $t_f = 54.77\,\text{h}$（约 2.28 天）。这一时间点标志着整个物料内部水分均降至安全范围内。

不同径向位置的水分脱除速率存在显著差异，模型通过对比不同半径处的浓度曲线验证了这一非均匀性。

表层与核心的差异：如图 2 所示（半对数坐标），表层（$r = 2.0\,\text{cm}$，黄色曲线）的水分在干燥初期迅速下降，远低于其他位置；而核心区域（$r = 0.0\,\text{cm}$，深紫色曲线）的浓度下降最为缓慢。

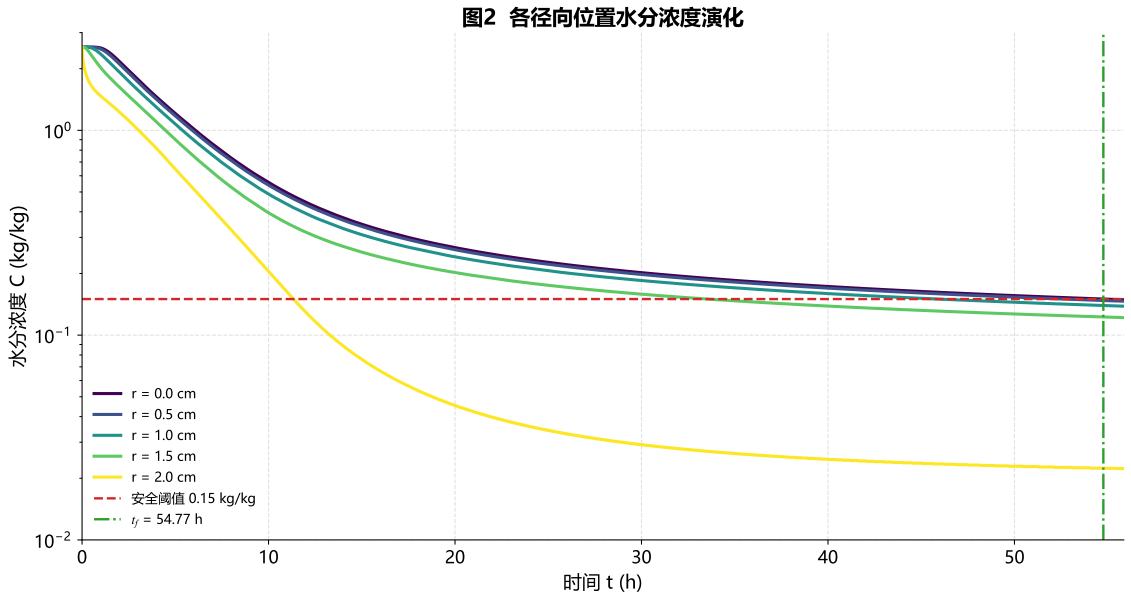


图 5-14: 各径向位置水分浓度演化

滞后效应验证：各层曲线在时间轴上呈现明显的分层排列，证实了水分迁移的“由外向内”机制。直到 $t_f = 54.77\,\mathrm{h}$（绿色竖线），最难以干燥的中心点才刚刚触及安全阈值，而此时表层水分已远低于该值。这一结果强调了在模型求解中必须以中心点作为控制变量，以防止出现“外干内湿”的现象。

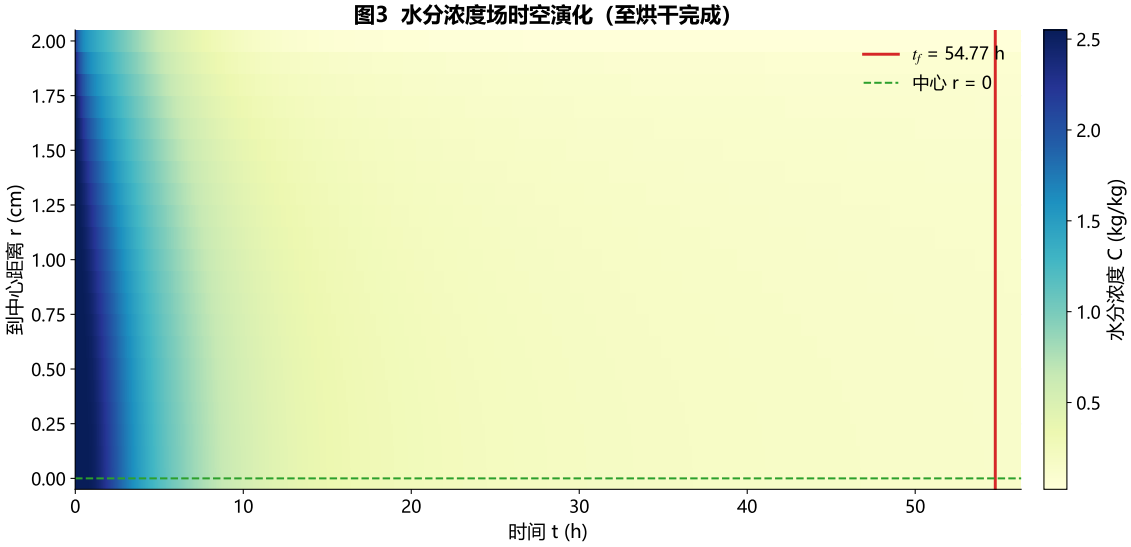


图 5-15: 水分浓度场时空变化

为了揭示干燥过程中内部传质的动态机制，模型进一步求解了全空间域内的浓度场分布。

梯度形成与推移：图 3 展示了水分浓度场 $C(r,t)$ 的时空演化热力图。在干燥初期（$t < 10\,\mathrm{h}$），物料边缘（$r = 2.0\,\mathrm{cm}$）迅速失水，导致径向方向上形成巨大的浓度梯度（深蓝色区域向浅黄色区域急剧过渡）。这种高梯度是驱动内部水分向外扩散的主要动力。

均质化过程：随着时间推移（$t > 30\,\mathrm{h}$），高浓度区域逐渐收缩并向中心退缩，整体颜色趋于均匀的浅黄色。这表明随着干燥深入，内部扩散阻力逐渐占据主导地位，浓度梯度逐渐平缓，干燥速率由恒速阶段转入降速阶段。

5.3.3 结果

表 5-5: 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6.0	1.0128	0.9835	0.8953	0.7455	0.5153
12.0	0.4456	0.4325	0.3922	0.3181	0.1293
18.0	0.2925	0.2853	0.2624	0.2184	0.0537
24.0	0.2332	0.2280	0.2118	0.1795	0.0358
30.0	0.2019	0.1978	0.1848	0.1585	0.0292
36.0	0.1823	0.1789	0.1678	0.1452	0.0260
42.0	0.1687	0.1657	0.1560	0.1358	0.0243
54.80	0.1489	0.1465	0.1385	0.1218	0.0223

5.4 问题四的分析和求解

5.4.1 模型的建立

针对问题四，考虑到药材在水分流失过程中会发生显著的体积收缩（即半径 R(t) 随时间减小），这是一个典型的移动边界问题（Stefan Problem 变体）。为了在固定的计算域内求解偏微分方程，我们引入 Landau 坐标变换。

1. Landau 坐标变换 (Landau Transformation)

在物理空间 (r, t) 中，药材的半径 R(t) 是随时间变化的函数，导致计算区域 $0 \leq r \leq R(t)$ 也是动态的。为了固定计算边界，我们引入无量纲坐标 ξ：

$$\xi = \frac{r}{R(t)} \tag{31}$$

其中：

r 为物理空间中的径向坐标 ($0 \leq r \leq R(t)$)。

R(t) 为时刻 t 时药材的瞬时半径。

ξ 为变换后的无量纲坐标，其取值范围固定为 [0, 1]。

ξ = 0 对应药材中心。

ξ = 1 对应药材表面（移动边界）。

通过该变换，原本在物理空间中随时间收缩的边界 r=R(t)，在 ξ 坐标系中被映射为固定的边界 $\xi = 1$。

2. 目标函数

我们的目标是求解药材完全干燥所需的时间 t_f。根据题目要求，当药材内部任意位置的水分浓度均低于安全阈值时，视为干燥完成。

定义干燥完成时间 t_f 为满足以下条件的最小时间值：

$$t_f = \min \left\{ t > 0 \mid \max_{\xi \in [0,1]} C(\xi, t) \leq 0.15 \right\}$$

或者表述为：寻找最小的 t，使得对于所有 $\xi \in [0, 1]$，都有：

$$C(\xi, t) \leq 0.15 \, (\text{kg/kg})$$

其中 $C(\xi, t)$ 是在变换坐标系下的水分浓度分布。

1. 水分扩散控制方程在 ξ 坐标系下，水分浓度 $C(\xi, t)$ 的演化方程为：

$$\left.\frac{\partial C}{\partial t}\right|_{\xi} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D(C, T) \frac{\partial C}{\partial \xi} \right) \tag{32}$$

此处忽略了收缩引起的对流项，或假设其影响包含在有效扩散系数中，主要考虑扩散主导机制。其中 $D(C, T)$ 为依赖于浓度和温度的水分扩散系数。

2. 热量传输控制方程同理，温度场 $T(\xi, t)$ 的演化方程为：

$$\left.\frac{\partial T}{\partial t}\right|_{\xi} = \frac{1}{R(t)^2 \xi \rho c_p} \frac{\partial}{\partial \xi} \left(\xi k \frac{\partial T}{\partial \xi} \right) \tag{33}$$

ρ 为密度，c_p 为比热容，k 为热导率。这些物性参数均通过附录 4 中的经验公式与 C 和 T 关联。

4.2 边界条件与初始条件

1. 表面边界条件 ($\xi = 1$) 在药材表面，水分和热量的传递遵循对流交换定律：

水分边界：

$$-D \frac{\partial C}{\partial r} = h_m (C_s - C_{eq}) \tag{34}$$

其中 h_m 为表面传质系数，C_s 为表面浓度，C_{eq} 为平衡含水率。

热量边界：

$$-k \frac{\partial T}{\partial r} = h (T_{air} - T_s) \tag{35}$$

其中 h 为表面传热系数，T_{air} 为环境空气温度，T_s 为表面温度。

2. 中心对称条件 ($\xi = 0$) 由于球体几何的对称性，在球心处通量为零：

$$\frac{\partial C}{\partial \xi} = 0, \quad \frac{\partial T}{\partial \xi} = 0$$

3. 初始条件 (t=0) 假设初始时刻药材内部状态均匀：

$$C(\xi, 0) = 2.55\,\text{kg/kg}, \quad T(\xi, 0) = 28^\circ\text{C}$$

4.3 环境控制策略与数值处理

1. 分段环境控制函数根据题目描述，干燥过程分为两个阶段，环境温度 $T_a(t)$ 和目标含水率采取分段控制：

$$(T_a(t), C_t) = \begin{cases} (T_t, \text{自由干燥}), & 0 < t \leq 7200\,\text{s} \\ (49.852^\circ\text{C}, 0.0488\,\text{kg/kg}), & t > 7200\,\text{s} \end{cases} \tag{36}$$

2. 物理量的 Clamp (截断) 处理为了保证数值计算的稳定性及物理意义的合理性，对计算过程中的浓度 C 和温度 T 进行限幅处理（Clamp Function）：

浓度限制：$C_{clamped} = \max(C, 0.15)$，确保计算出的浓度不低于安全阈值。

温度限制：$T_{clamped} = \text{clip}(T, 301.15\,\text{K}, 323.40\,\text{K})$，即限制温度在 28°C 至 50.25°C 之间，防止数值振荡导致温度越界。

问题 4 用到的经验公式

$$\rho = 760 + 90C \tag{37}$$

$$c_p = 1850 + 2150 \cdot \frac{C}{C+1} \tag{38}$$

$$k = 0.12 + 0.20 \cdot \frac{C+1}{0.30} e^{\frac{3850}{T}} \tag{39}$$

$$D = 4.2 \times 10^{-4} e^{-\frac{C}{0.30}} e^{\frac{3850}{T}} \tag{40}$$

其中 ρ 为密度（单位：kg/m^3），c_p 为比热容（单位：$\text{J/(kg} \cdot \text{K)}$），k 为热传导系数（单位：$\text{W/(m} \cdot \text{K)}$），D 为水分浓度扩散系数（单位：$\text{m}^2/\text{s}$），C 为药材水分浓度（单位：kg/kg），T 为药材温度（单位：K）。

5.4.2 模型的求解

本节采用有限差分法对模型进行数值求解。空间离散取网格数 $N = 20$ ($\Delta\xi = 0.05$)，时间步长设定为 $\Delta t = 0.5\,\text{s}$。在数值格式上，内部节点采用显式差分格式，表面半格节点采用隐式处理以增强边界稳定性。针对半径 $R(t)$ 的动态变化，利用 PCHIP 插值算法重构边界曲线，并在计算过程中实时校验 CFL 条件及最小半径阈值 $R_{\min} = 1.198\,\text{cm}$ 以确保数值收敛。终止判据基于中心点 ($\xi = 0$) 达标时刻的线性插值精确定位，并在此基础上额外延长 1 h 仿真时长以验证终态稳定性，同时通过质量守恒核算校验计算精度。

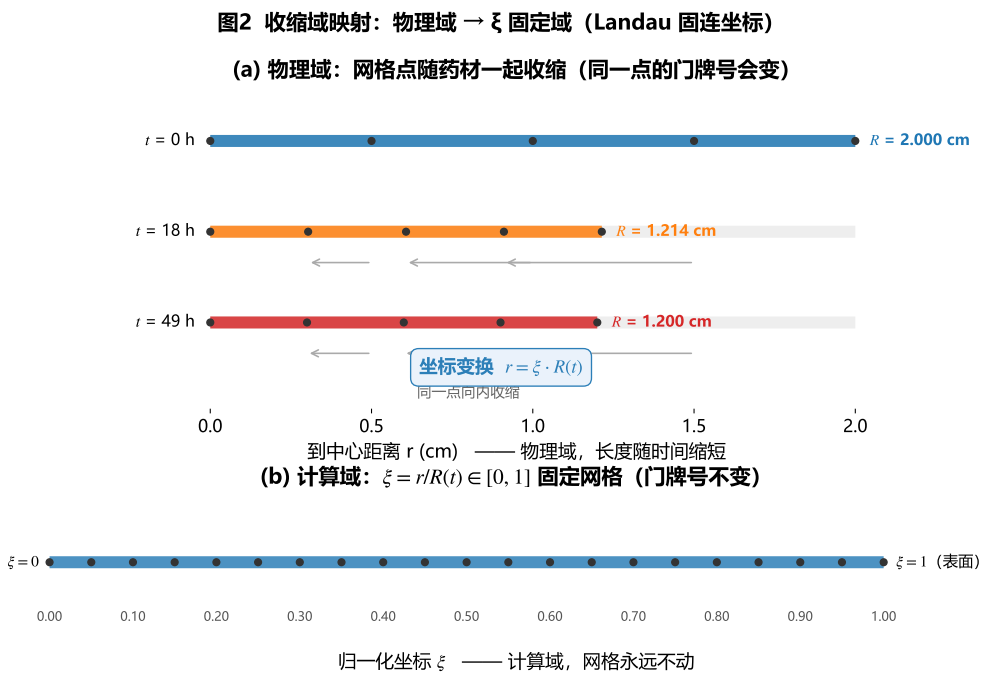


图 5-16: 收缩域映射

数值求解策略：Landau 坐标变换针对药材在干燥过程中发生的显著体积收缩现象（半径从 R_0 =2.000 cm 收缩至 1.200 cm），直接采用固定空间网格的有限差分法会导致计算域随时间变化，产生较大的截断误差。为此，本模型引入了 Landau 坐标变换（如图 2 所示）：

- **物理域映射**：将随时间 t 变化的物理半径 $r \in [0, R(t)]$ 映射为固定的归一化计算域 $\xi \in [0, 1]$。变换公式定义为：

$$\xi = \frac{r}{R(t)} \tag{41}$$

- **网格处理**：在计算域 ξ 上划分固定网格（即“门牌号不变”），从而避免了移动边界问题。通过链式法则推导出的包含收缩项 $\frac{\dot{R}}{R}\xi$ 的对流-扩散方程，能够精确描述水分在收缩介质中的传输规律。

干燥动力学特性分析图 3 展示了中心与表面水分浓度 C 随时间 t 的演化曲线。

- **表面响应迅速**：药材表面（$\xi = 1$）水分浓度在初期迅速下降，约在 $t_s = 11.40$ h 时即达到安全阈值 0.15 kg/kg。
- **内部滞后效应**：由于内部扩散阻力，中心点（$\xi = 0$）作为“最不利点”，其水分下降显著滞后。直至 $t_f = 49.45$ h（约 2.06 day），中心浓度才降至安全阈值以下。
- **结论**：整个干燥过程由内部扩散控制，**烘干完成时间取决于中心点的达标时间**，即 $t_f \approx 49.45$ h。此时药材体积缩至初始的 36.0%（$(R/R_0)^2 \approx 0.360$）。

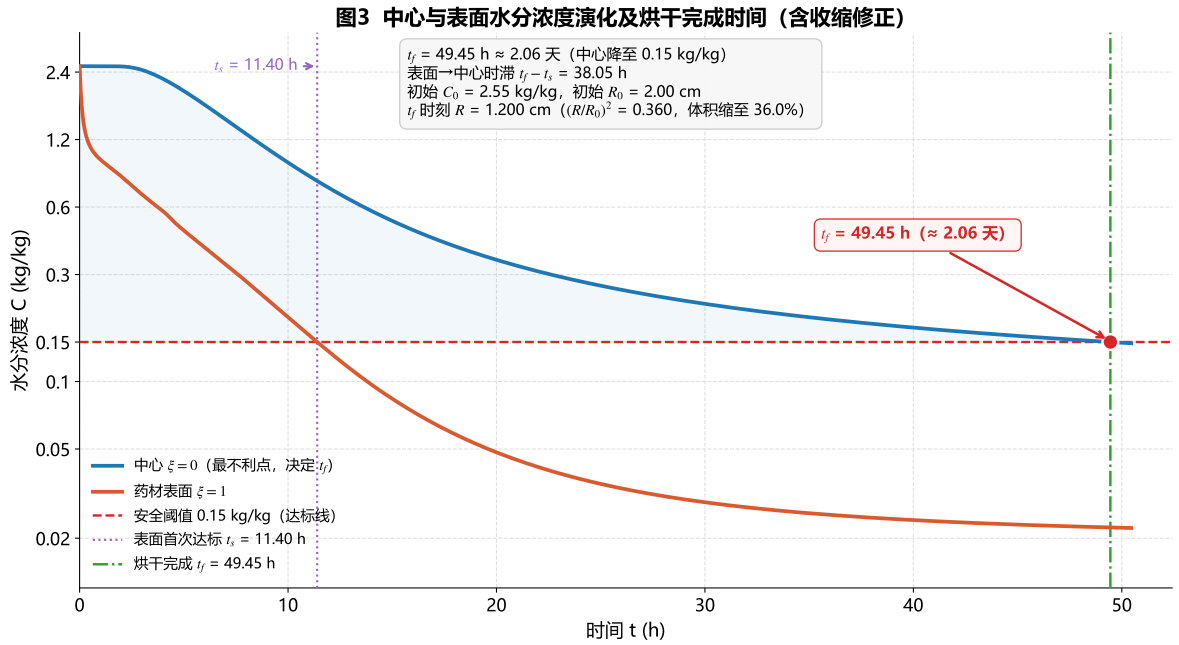


图 5-17: 中心与表面水分浓度演化及烘干完成时间

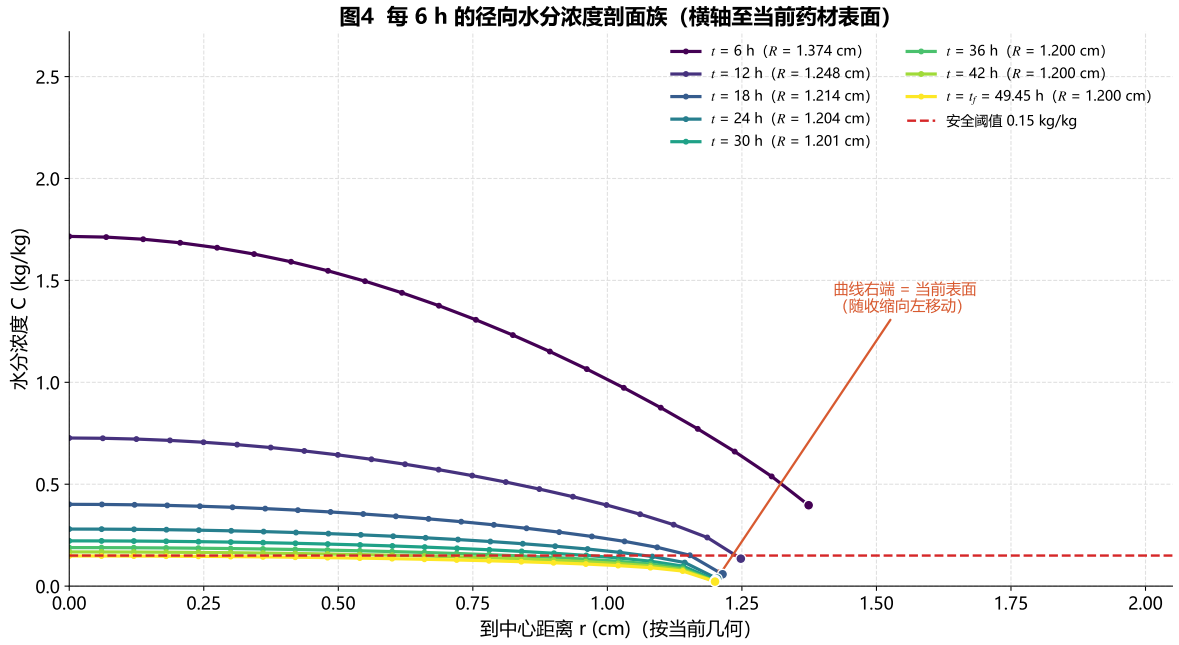


图 5-18: 每 6h 的径向水分浓度剖面

径向剖面演化规律图 4 进一步展示了不同时刻径向水分浓度的空间分布族。

- **梯度变化**：随着干燥进行，径向浓度梯度逐渐减小。
- **边界移动**：曲线右端点代表当前药材表面位置，随时间向左移动，直观反映了药材体积的收缩过程。

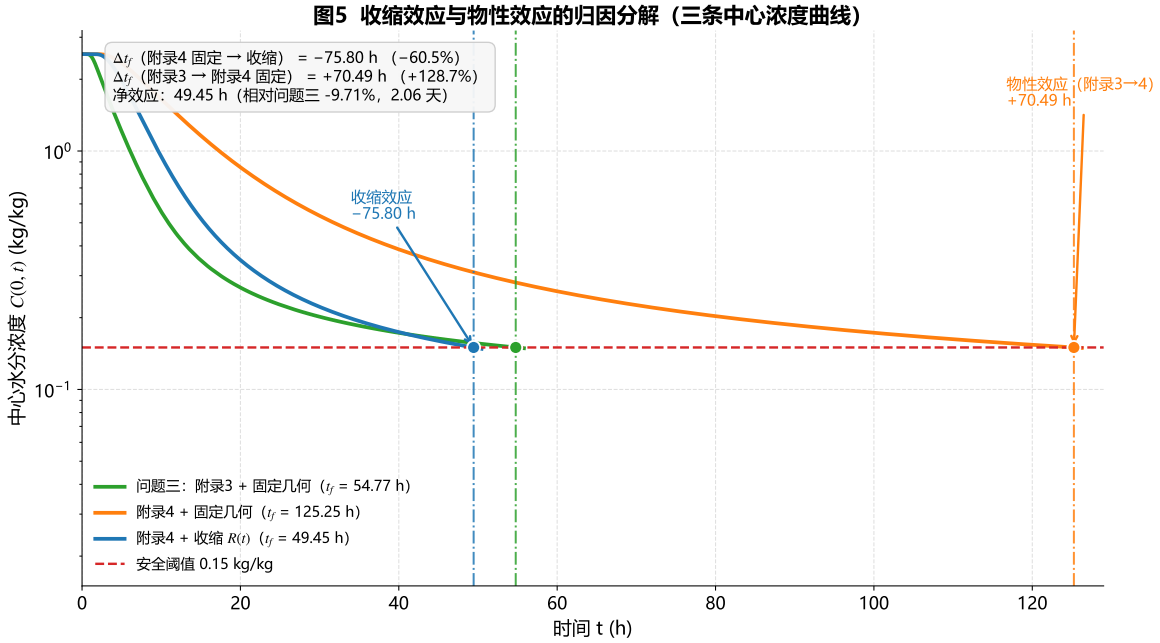


图 5-19: 收缩效应与物性效应的归因分解

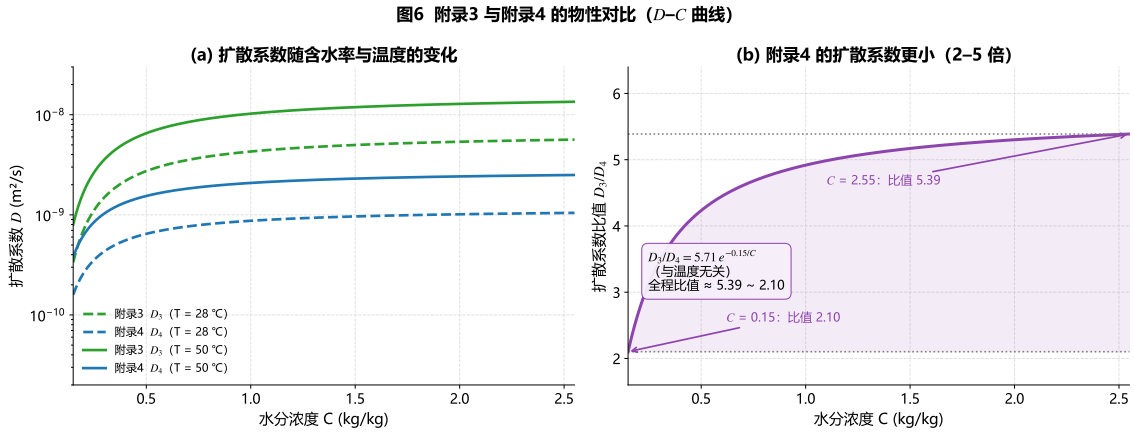


图 5-20: 附录 3 与附录 4 的物性对比

收缩效应与物性效应的归因分解为了量化不同因素对干燥时间的影响，我们进行了控制变量分析（如图 5 和图 6 所示）。

- **物性差异**：对比附录 3 与附录 4 的扩散系数 $D-C$ 曲线（图 6），附录 4 的扩散系数显著更小（约为附录 3 的 1/2 至 1/5）。若忽略收缩，仅考虑物性变化，干燥时间将大幅延长至 125.25 h（增加 70.49 h）。
- **收缩效应**：引入体积收缩修正后，由于扩散路径缩短，干燥时间反而缩短了 75.80 h（−60.5%）。
- **净效应**：两种效应叠加后，最终确定的烘干时间为 49.45 h。这表明虽然物性参数导致扩散变慢，但几何尺寸的收缩极大地加速了干燥进程，起到了主导作用。

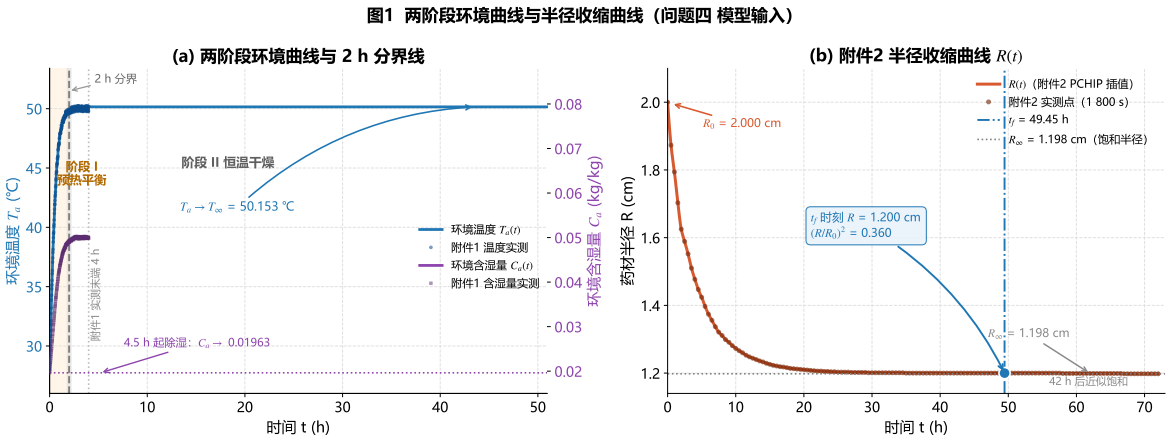


图 5-21: 两阶段环境曲线与半径收缩曲线

环境输入与边界条件图 1 展示了模型求解所需的输入条件。环境温度 $T_a(t)$ 呈现两阶段特征：前 2 h 为预热平衡阶段，随后进入 50.153 °C 的恒温干燥阶段。半径收缩曲线 $R(t)$ 采用 PCHIP 插值拟合实测数据，确保了边界移动的准确性。

5.4.3 结果

表 5-6: 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2.0
6.0	1.7159	1.6292	1.3759	0.9730	0.3970
12.0	0.7267	0.6942	0.5981	0.4388	0.1348
18.0	0.4013	0.3867	0.3426	0.2649	0.0583
24.0	0.2800	0.2713	0.2447	0.1958	0.0368
30.0	0.2220	0.2160	0.1971	0.1612	0.0289
36.0	0.1890	0.1843	0.1695	0.1408	0.0254
42.0	0.1678	0.1639	0.1516	0.1274	0.0236
49.45	0.1481	0.1449	0.1348	0.1146	0.0222

六、 模型的分析与检验

6.1 药材含水量外推分析

6.1.1 灵敏度分析

在模型假设中，我们约定当药材含水率达到 0.15 kg/kg 时，含水率保持 0.15 kg/kg 不变。但在真实的情况中，含水率 C 必然会跌破 0.15kg/kg，因此，为验证模型的稳健性，本文对关键参数含水率下界 Cprop 进行灵敏度分析。

6.1.2 扰动设计

由于本题阈值恰好为 0.15 ，±10% 小扰动不足以辨别出干壳效应，±30% 的幅度可充分体现，因此以 C_{prop}=0.15kg/kg，上下浮动 ±10%,±20%,±30%, 分别求烘干时间。

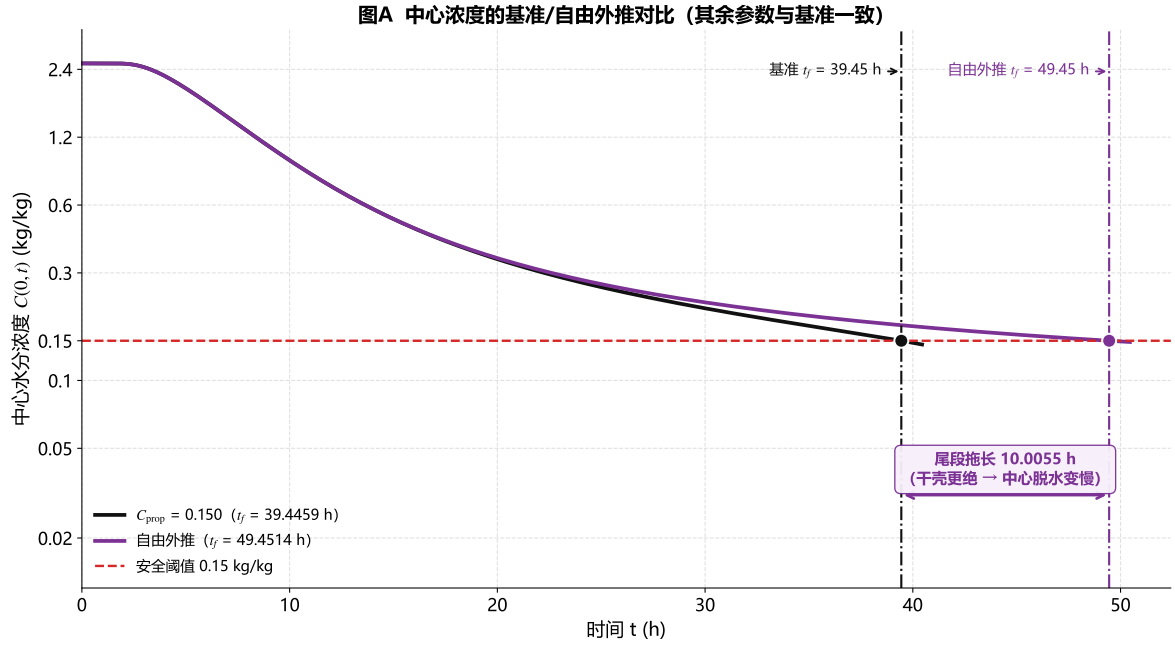


图 6-22: 中心浓度的基准/自由外推对比

这幅图说明，尾段处理方式对干燥终点预测影响很大。基准模型预测约 39.45 h 达到安全水分，而自由外推模型预测约 49.45 h，比基准晚约 10.01 h。因此，若采用自由外推，干燥时间会显著延长，基准结果可能偏乐观；自由外推结果更保守，反映了干壳阻力增强后中心脱水变慢的实际风险。

6.1.3 结果分析

当烘干时间自由外推时，中心水分浓度下降到 0.15 后就基本保持稳定。因此，我们假定当含水率低于 0.15 时含水率就保持不变基本合理。

6.2 基于传质毕渥数的对流系数灵敏度分析

6.2.1 灵敏度分析

对流换热系数 h 与对流传质系数 h_m 是本题中唯一全部沿用的参数，并且它出现在边界条件中，可能直接影响问题的分析。因此，为了验证模型的稳健性，对 h、h_m 浮动

±10%,±20%,±30% 分析。

6.2.2 图表分析

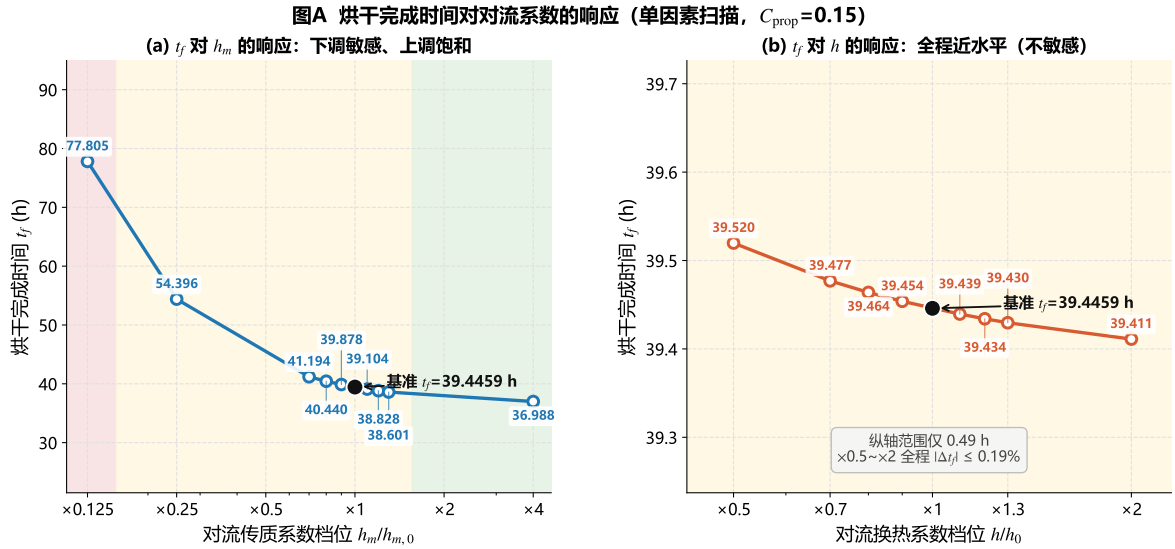


图 6-23: 烘干完成时间对对流系数的响应

子图 (a)：t_f 对 h_m 呈”下调敏感、上调饱和”特征，h_m 减半时 t_f 从 39.45 h 飙升至 41.194 h，下调至 1/8 时高达 77.805 h；上调时则趋于饱和（×4 仅降至 36.988 h）。

子图 (b)：t_f 对 h 几乎不敏感，×0.5~×2 全程 $|t_f| \leq 0.19\%$，曲线近乎水平。

传质毕渥数 Bi_m 表示内部扩散阻力与表面对流传质阻力的比值。其计算公式为：

$$Bi_m = \frac{h_m R}{D} \tag{42}$$

其中，h_m 为对流传质系数，D 为水分扩散系数。根据题目给定参数估算：$Bi_m \approx \frac{8\times10^{-7}\times0.02}{7\times10^{-9}} \approx 23$

$Bi_m \approx 23$ 是一个较大的数值（通常 $Bi_m > 10$ 即视为大数值）。这说明表面的对流传质阻力相对于内部扩散阻力非常小，表面水分能迅速蒸发，干燥速率主要受限于水分在药材内部的扩散速度。这也进一步证实了水分浓度在径向上存在巨大差异，必须通过求解扩散方程（PDE）来获取内部水分分布。

6.2.3 结果分析

图 6-23 显示，t_f 对 h 不敏感（×0.5 ×2 变化 <0.19%），对 h_m 呈”上调饱和、下调敏感”的非对称响应：×4 仅 −6.2%（趋于内部扩散极限 36.99 h），×0.5 时 +4.4%，×0.125 时高达 +97.2%。机制上，$Bi_m = h_m R/D$ 上调至 25.6 时内部扩散始终主导、收益饱和；降至 0.8 时表面阻力与内部阻力相当、t_f 近乎翻倍——Bi_m~ 1 即假设的失效边界。沿用附录 2 参数具有稳健性，但低风速情况须重新计入表面阻力。

七、 模型的评价与推广

7.1 模型的评价

1. 优点

(1) 物理机制刻画深入，符合实际干燥过程本模型没有简单地将干燥过程视为单纯的扩散，而是建立了热质耦合传输方程, 充分考虑了物性参数（密度、比热容、热导率、扩散系数）随温度 T 和浓度 C 的非线性变化关系。

(2) 巧妙运用 Landau 变换解决移动边界难题针对问题四中因水分流失导致的药材体积收缩（移动边界）问题，模型引入了 Landau 坐标变换 ($\xi = r/R(t)$)。这一数学技巧成功地将随时间变化的非固定计算域映射为固定的单位区间 [0, 1]，避免了复杂的网格重构（Remeshing），极大地降低了数值求解的难度，同时保证了计算过程的稳定性。

2. 缺点

尽管本文建立的圆柱坐标系下的一维非稳态热质耦合模型能够较好地模拟药材烘干过程中的温度场与浓度场分布，但在物理机制的完整性上仍存在一定局限，主要体现在以下方面：

- **忽略了表面水分蒸发的潜热效应：**在构建药材表面的能量守恒边界条件（第三类边界条件）时，本文主要考虑了热风对表面的对流加热通量 $h(T_a - T_s)$ 与内部传导热通量的平衡。然而，在实际烘干过程中，药材表面水分的剧烈蒸发会吸收大量的汽化潜热（$L_v \cdot j_m$）。本文模型未显式地在表面能量平衡方程中减去这一项潜热通量。
- **内部应力与裂纹未予考虑：**模型仅考虑了水分和热量的传输，忽略了干燥过程中因含水率梯度产生的内部应力。在实际快速干燥中，应力可能导致药材表面开裂，从而改变传质路径。

八、 模型的改进

针对上述不足，未来可从以下维度对模型进行修正与深化：

8.1 缺点 1 的改进

1. **引入完整的表面能量耦合边界条件：**在未来的研究中，应严格遵循能量守恒定律，修正表面边界条件方程。将水分蒸发带走的潜热通量纳入考量，建立如下形式的能量平衡方程：

$$-k\frac{\partial T}{\partial r}\bigg|_{r=R} = h(T_a(t) - T(R,t)) - L_v \cdot h_m \rho(C(R,t) - C_{eq})$$

其中 L_v 为水的汽化潜热，h_m 为对流传质系数。通过引入这一项，可以更准确地描述“蒸发冷却”效应对药材表面温度的抑制作用，从而提高模型在高温快速干燥阶段的拟合精度。

2. **考虑变物性参数的动态反馈机制：**目前的模型虽然引入了随含水率变化的密度 $\rho(C)$ 和比热容 $c_p(C)$，但导热系数 k 和扩散系数 D 的变化规律可能仍较为简化。

改进模型可以进一步结合实验数据，建立 $k(T,C)$ 和 $D(T,C)$ 的双变量非线性函数关系，以更真实地反映热质传递过程中的交叉耦合效应。

8.2 缺点 2 的改进

1. **建立流变学本构方程：**将药材视为粘弹性体（Viscoelastic Material），引入 Maxwell 或 Kelvin-Voigt 模型来描述其应力-应变关系。
2. **耦合应力扩散方程：**在原有的质量传输方程中增加一项由应力梯度驱动的“压力流”项（Darcy Flow）：

$$J_{total} = -D\nabla C - \frac{K}{\mu}\nabla P_{stress}$$

其中 P_{stress} 是由局部含水率差异引起的毛细管压力和膨胀压力的总和。

3. **失效准则判据：**设定一个临界应力阈值 $\sigma_{critical}$。当计算出的内部张力 $\sigma > \sigma_{critical}$ 时，动态修正扩散系数 D（模拟裂纹产生导致扩散加速）或改变边界条件。

参考文献

[1] 千问 (Qwen), Qwen-Max (基于通义 Qwen 最新闭源模型), 阿里巴巴 (Alibaba Cloud), 2025-09-10

[2] WorkBuddy, 云端智能体版（CodeBuddy Agent）, 腾讯（Tencent）, 2026-09-10

[3] DeepSeek, DeepSeek-R1-0528, 深度求索（DeepSeek）, 2026-09-10

[4] 李冰. 药材烘干器自动控制系统设计与实现 [J]. 电子制作,2024,32(20):39-41+101.DOI:10.16589/j.cnki.cn11-3571/tn.2024.20.027.

[5] 高健铭, 吕锡亮. 线性反问题的 Crank-Nicolson 正则化方法 [J]. 数学杂志,2026,46(3):165-176.DOI:10.13548/j.sxzz.20260126.001.

[6] 王兴波, 钟志华. 求解周期性三对角方程组的广义 Thomas 算法 [J]. 计算力学学报,2004,(1):73-76.

[7] 王圆梅. 偏微分控制系统有限差分格式的稳定性研究 [D]. 北方工业大学,2025.DOI:10.26926/d.cnki.gbfgu.2025.000972.

[8] 李春雷, 王亚男, 葛仁磊, 等. 非线性模型的抗差最小二乘法及其在圆心拟合中的应用 [J]. 山东化工,2023,52(13):178-181.DOI:10.19319/j.cnki.issn.1008-021x.2023.13.043.

[9] 邱敏, 程秀俊. 诺伊曼边界条件下分数阶次扩散方程的紧差分格式 [J]. 浙江理工大学学报 (自然科学版),2021,45(2):234-241.

[10] 田旭, 黄湘友. 非耦合压缩 Landau 态 [J]. 物理学报,1998,(5):15-20.

附录 A: 支撑材料清单

AI工具使用详情.pdf	PDF 文件	207 KB
dry_model.py	Python 源文件	7 KB
generate_figures.py	Python 源文件	8 KB
generate_figures2.py	Python 源文件	6 KB
generate_figures3.py	Python 源文件	3 KB
generate_figures4.py	Python 源文件	10 KB
result1.xlsx	Microsoft Excel 工作表	308 KB
result2.xlsx	Microsoft Excel 工作表	2,118 KB
result3.xlsx	Microsoft Excel 工作表	290 KB
result4.xlsx	Microsoft Excel 工作表	311 KB
sensitivity_q4.py	Python 源文件	5 KB
sensitivity_q4_hhm.py	Python 源文件	3 KB
solve_problem1.py	Python 源文件	3 KB
solve_problem2.py	Python 源文件	3 KB
solve_problem3.py	Python 源文件	4 KB
solve_problem4.py	Python 源文件	7 KB

附录 B:Python 代码

```
"""
问题1：预热平衡阶段药材温度与水分浓度的变化规律。

模型：圆柱（一维径向）非稳态导热 + 非稳态扩散双场耦合
      c_p  T/ t = k  ²T            （忽略汽化潜热，量级占比 < 1%，见论文）
     C/ t      = (1/r)  / r ( r D(C)  C / r )
边界：r = 0 对称；r = R 第三类（对流换热 / 对流传质）
物性：附录2 常物性，D = 7e-9 exp(-0.89/C )
格式：全隐式 + 追赶法，无条件稳定
"""

import os
import numpy as np

from dry_model import (
    T0_C, C0, BASE_DIR,
    PARAMS_Q1, Grid, tdma, iface_mean,
    make_env, sample_indices, interp_to_distances,
    save_result, TARGET_DISTS,
)

# =========
# 配置
# =========
N = 160              # 径向网格数
DT = 0.1             # 时间步长 (s)
T_END = 1800.0       # 模拟时长 (s)
SAMPLE_DT = 1.0      # 输出采样间隔 (s)

P = PARAMS_Q1
grid = Grid(N)
Ta, Ca, _ = make_env()


# =======
# 时间推进：水分场与温度场均用全隐式格式推进
# =======
def solve():
    """返回 (时间数组, 温度矩阵, 水分浓度矩阵)。"""
    n = N + 1
    dr = grid.dr
    r_int, r_half, r = grid.r_int, grid.r_half, grid.r

    T = np.full(n, T0_C)
    C = np.full(n, C0)

    times, steps = sample_indices(T_END, DT, SAMPLE_DT)
    T_out = np.zeros((len(steps), n))
    C_out = np.zeros((len(steps), n))
    T_out[0], C_out[0] = T, C
    out_idx = 1

    nsteps = int(round(T_END / DT))
    for step in range(1, nsteps + 1):
        t_new = step * DT

        # ---------------- 水分扩散（隐式） ----------------
        a = np.zeros(n); b = np.zeros(n); c = np.zeros(n); d = np.zeros(n)
        D_node = P.D(C)
        for i in range(1, N):
            ri = r_int[i - 1]
            rhp = r_half[i - 1]
            rhm = r_half[i - 2] if i - 2 >= 0 else r_half[0]
            D_ip = iface_mean(D_node[i], D_node[i + 1])
            D_im = iface_mean(D_node[i -
```

```
            ↪  1], D_node[i])
        cp_ = DT * rhp * D_ip / (ri
            ↪ * dr ** 2)
        cm_ = DT * rhm * D_im / (ri
            ↪ * dr ** 2)
        a[i] = -cm_
        b[i] = 1.0 + cp_ + cm_
        c[i] = -cp_
        d[i] = C[i]

    # 轴心：对称条件，等价于虚拟节点法
    b[0] = 1.0 + 4.0 * D_node[0] *
        ↪ DT / dr ** 2
    c[0] = -4.0 * D_node[0] * DT /
        ↪ dr ** 2
    d[0] = C[0]

    # 表面：第三类边界，半格控制体，隐式处理
    A_R, A_in, V_h = grid.A_R, grid.
        ↪ A_inner, grid.V_half
    D_s = D_node[N]
    a[N] = -D_s * A_in / dr
    b[N] = V_h / DT + P.hm * A_R +
        ↪ D_s * A_in / dr
    d[N] = V_h / DT * C[N] + P.hm *
        ↪ A_R * Ca(t_new)
    C = tdma(a, b, c, d)

    # ---------------- 温度场（隐式） ----------------
    a = np.zeros(n); b = np.zeros(n)
        ↪ ; c = np.zeros(n); d = np
        ↪ .zeros(n)
    alpha = P.k(C)[0] / (P.rho(C)[0]
        ↪  * P.cp(C)[0])
    for i in range(1, N):
        ri = r_int[i - 1]
        cp_ = alpha * DT * (1.0 / dr
            ↪  ** 2 + 1.0 / (2.0 *
            ↪ ri * dr))
        cm_ = alpha * DT * (1.0 / dr
            ↪  ** 2 - 1.0 / (2.0 *
            ↪ ri * dr))
        a[i] = -cm_
        b[i] = 1.0 + cp_ + cm_
        c[i] = -cp_
        d[i] = T[i]

    b[0] = 1.0 + 4.0 * alpha * DT /
        ↪ dr ** 2
    c[0] = -4.0 * alpha * DT / dr **
        ↪  2
    d[0] = T[0]

    rho_cp = P.rho(C)[N] * P.cp(C)[N
        ↪ ]
    a[N] = -P.k(C)[N] * A_in / dr
    b[N] = rho_cp * V_h / DT + P.h *
        ↪  A_R + P.k(C)[N] * A_in /
        ↪  dr
    d[N] = rho_cp * V_h / DT * T[N]
        ↪ + P.h * A_R * Ta(t_new)
    T = tdma(a, b, c, d)

    # ---------------- 采样 ----------------
    if out_idx < len(steps) and step
        ↪  == steps[out_idx]:
        T_out[out_idx], C_out[
            ↪ out_idx] = T, C
        out_idx += 1

return times, T_out, C_out


# =======
# 主流程
# =======
times, T_out, C_out = solve()

# 插值到题面要求的到中心距离
T_grid = np.array([interp_to_distances(
```

```
    ↪ grid.r, T_out[i], TARGET_DISTS)
                    for i in range(len(
                        ↪ times))])
C_grid = np.array([interp_to_distances(
    ↪ grid.r, C_out[i], TARGET_DISTS)
                    for i in range(len(
                        ↪ times))])

save_result(
    os.path.join(BASE_DIR, "result1.xlsx
        ↪ "),
    [("温度", times, T_grid, "时间"), ("
        ↪ 水分浓度", times, C_grid, "时
        ↪ 间")],
)

# ======
# 表1、表2
# ======
TABLE_TIMES = [100, 300, 600, 900, 1200,
    ↪  1500, 1800]
TABLE_DISTS = [0.0, 0.5, 1.0, 1.5, 2.0]


def col(d):
    return int(np.argmin(np.abs(
        ↪ TARGET_DISTS - d)))


def row(t_val):
    return int(np.argmin(np.abs(times -
        ↪ t_val)))


for label, data in [("表1  30分钟内药材
    ↪ 的温度 (°C)", T_grid),
                    ("表2  30分钟内药材
                        ↪ 的水分浓度 (
                        ↪ kg/kg)",
                        ↪ C_grid)]:
    print(f"\n{label}")
    print(f"{'时间/s':<12}" + "".join(f"
        ↪ {d}cm".rjust(12) for d in
        ↪ TABLE_DISTS))
    print("-" * 72)
    for tv in TABLE_TIMES:
        print(f"{tv:<12}" + "".join(f"{
            ↪ data[row(tv), col(d)
            ↪ ]:>12.4f}" for d in
            ↪ TABLE_DISTS))

print(f"\n[输出] result1.xlsx: {len(
    ↪ times)} 行 x {len(TARGET_DISTS)}
    ↪ 列")
```

Listing 1: 主程序代码

```
"""
问题2：整个烘干过程药材温度与水分浓度的
    ↪ 变化规律（前 3 小时）。

模型：圆柱（一维径向）非稳态导热 + 非稳
    ↪ 态扩散双场耦合
      (C) c_p(C)  T/ t = (1/r)  / r ( r
        ↪ k(C)  T/ r )
      C/ t             = (1/r)  / r ( r
        ↪ D(C,T)  C/ r )
物性：附录3 变物性
阶段：预热平衡（t < t_c）→ 恒温干燥（t
    ↪  t_c），切换时刻由附件1 标定为 2
    ↪ h。
     环境温湿度在 t_c 后趋于恒定，边界条
        ↪ 件由强时变退化为准稳态
    Robin 条件，参数组随阶段切换。
格式：内部节点显式，表面半格节点隐式（解
    ↪ 除表面刚性引起的稳定性限制）
"""

import os
import numpy as np

from dry_model import (
    C0, T0_C, BASE_DIR, T_SWITCH,
        ↪ T_INF_C, C_INF,
```

```
    Grid, harm_mean, iface_mean,
        ↪ make_env, sample_indices,
    interp_to_distances, save_result,
        ↪ TARGET_DISTS, params_at,
)

# =====
# 配置
# =====
N = 20              # 径向网格数
DT = 0.5            # 时间步长 (s)
T_END = 10800.0     # 模拟时长, 3 h (s)
SAMPLE_DT = 1.0     # 输出采样间隔 (s)

grid = Grid(N)
Ta, Ca, _ = make_env()


# =======
# 时间推进
# =======
def solve():
    """返回 (时间数组, 温度矩阵, 水分浓
        ↪ 度矩阵)。"""
    n = N + 1
    dr = grid.dr
    r_int, r_half = grid.r_int, grid.
        ↪ r_half
    A_R, A_in, V_h = grid.A_R, grid.
        ↪ A_inner, grid.V_half
    vols = grid.cell_volumes()

    T = np.full(n, T0_C)
    C = np.full(n, C0)

    times, steps = sample_indices(T_END,
        ↪  DT, SAMPLE_DT)
    T_out = np.zeros((len(steps), n))
    C_out = np.zeros((len(steps), n))
    T_out[0], C_out[0] = T, C

    water_init = float(np.sum(C * vols))
    evap_cum = 0.0
    out_idx = 1

    nsteps = int(round(T_END / DT))
    for step in range(1, nsteps + 1):
        t_new = step * DT
        P = params_at(t_new - 0.5 * DT)
            ↪       # 阶段物性取半步前时
            ↪ 刻

        # ---------------- 物性 (滞后取
            ↪ 旧时刻值)
            ↪ ----------------
        rho_n = P.rho(C)
        cp_n = P.cp(C)
        k_n = P.k(C)
        D_n = P.D(C, T + 273.15)

        k_h = harm_mean(k_n[:-1], k_n
            ↪ [1:])
        D_h = iface_mean(D_n[:-1], D_n
            ↪ [1:])   # 浓度连续, 界面
            ↪ 扩散系数取线性插值

        # ---------------- 各节点散度项
            ↪ ----------------
        divT = np.zeros(n)
        divC = np.zeros(n)

        # 轴心: 对称条件
        divT[0] = 4.0 * k_h[0] * (T[1] -
            ↪  T[0]) / dr ** 2
        divC[0] = 4.0 * D_h[0] * (C[1] -
            ↪  C[0]) / dr ** 2

        # 内部节点: 界面通量差分
        qT = r_half * k_h * (T[1:] - T
            ↪ [:-1]) / dr
        qC = r_half * D_h * (C[1:] - C
            ↪ [:-1]) / dr
        divT[1:N] = (qT[1:] - qT[:-1]) /
            ↪  (r_int * dr)
```

```
        divC[1:N] = (qC[1:] - qC[:-1]) / (r_int * dr)

        # ---------------- 内部节点推进（显式） ----------------
        C_prev = C.copy()
        T_new = T + DT * divT / (rho_n * cp_n)
        C_new = np.concatenate([np.maximum(C[:N] + DT * divC[:N], 0.0), [C[N]]])

        # ---------------- 表面节点：半格控制体 + 隐式 ----------------
        # 能量：  cp V_h (T_N^new - T_N)/Δt = h A_R (Ta - T_N^new) - k A_in/Δr (T_N^new - T_{N-1})
        Ta_v = Ta(t_new)
        rho_cp_s = rho_n[N] * cp_n[N]
        T_new[N] = ((rho_cp_s * V_h / DT * T[N]
                     + P.h * A_R * Ta_v
                     + k_h[-1] * A_in / dr * T_new[N - 1])
                    / (rho_cp_s * V_h / DT + P.h * A_R + k_h[-1] * A_in / dr))

        # 水分：  V_h (C_N^new - C_N)/Δt = hm A_R (Ca - C_N^new) - D A_in/Δr (C_N^new - C_{N-1})
        Ca_v = Ca(t_new)
        C_new[N] = ((V_h / DT * C[N]
                     + P.hm * A_R * Ca_v
                     + D_h[-1] * A_in / dr * C_new[N - 1])
                    / (V_h / DT + P.hm * A_R + D_h[-1] * A_in / dr))

        # 累积蒸发量：边界传质通量积分
        evap_cum += P.hm * (C_prev[N] - Ca_v) * A_R * DT

        T, C = T_new, C_new

        if out_idx < len(steps) and step == steps[out_idx]:
            T_out[out_idx], C_out[out_idx] = T, C
            out_idx += 1

    # ---------------- 质量守恒校核 ----------------
    water_final = float(np.sum(C * vols))
    loss = water_init - water_final
    rel = abs(loss - evap_cum) / water_init
    print("=" * 60)
    print("质量守恒检查")
    print("=" * 60)
    print(f"初始总水量:      {water_init:.6e}")
    print(f"最终总水量:      {water_final:.6e}")
    print(f"内部水量减少:    {loss:.6e}")
    print(f"边界累积蒸发:    {evap_cum:.6e}")
    print(f"相对误差:        {rel:.3e}")

    return times, T_out, C_out


# =======
# 主流程
```

```python
# =======
times, T_out, C_out = solve()

T_grid = np.array([interp_to_distances(grid.r, T_out[i], TARGET_DISTS)
                   for i in range(len(times))])
C_grid = np.array([interp_to_distances(grid.r, C_out[i], TARGET_DISTS)
                   for i in range(len(times))])

save_result(
    os.path.join(BASE_DIR, "result2.xlsx"),
    [("温度", times, T_grid, "时间"), ("水分浓度", times, C_grid, "时间")],
)

# =======
# 表3、表4
# =======
TABLE_HOURS = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
TABLE_DISTS = [0.0, 0.5, 1.0, 1.5, 2.0]


def col(d):
    return int(np.argmin(np.abs(TARGET_DISTS - d)))


def row(t_val):
    return int(np.argmin(np.abs(times - t_val)))


for label, data in [("表3  3小时内药材的温度 (°C)", T_grid),
                    ("表4  3小时内药材的水分浓度 (kg/kg)", C_grid)]:
    print(f"\n{label}")
    print(f"{'时间/h':<12}" + "".join(f"{d}cm".rjust(12) for d in TABLE_DISTS))
    print("-" * 72)
    for th in TABLE_HOURS:
        mark = "   ← 阶段切换" if abs(th * 3600 - T_SWITCH) < 1e-6 else ""
        print(f"{th:<12.1f}" + "".join(f"{data[row(int(th * 3600)), col(d)]:>12.4f}" for d in TABLE_DISTS) + mark)

print(f"\n[环境] 阶段切换 t_c = {T_SWITCH/3600:.1f} h; "
      f"恒温渐近 Tϖ = {T_INF_C:.3f} °C, Cϖ = {C_INF:.5f} kg/kg")
print(f"[输出] result2.xlsx: {len(times)} 行 x {len(TARGET_DISTS)} 列")
```

Listing 2: 主程序代码

```
"""
问题3：确定药材烘干所需时间（各处水分浓度低于 0.15 kg/kg）。

模型：与问题2 同一套圆柱双场耦合模型，延长至烘干完成。
阶段：预热平衡（t < t_c = 2 h）→ 恒温干燥（t  t_c），参数组随阶段切换。
判据：以药材中心 r = 0 的水分浓度首次降至 C_SAFE = 0.15 kg/kg 作为
  烘干完成时刻 t_f —— 中心是最后达标的部位。
  物性函数的数值保护下界取 1e-4，与判据阈值解耦，
  避免阈值处扩散系数被人为抬高而系统性提前 t_f。
```

```
格式：内部节点显式，表面半格节点隐式。
"""

import os
import numpy as np

from dry_model import (
    C0, T0_C, C_SAFE, C_CLIP_LO, BASE_DIR, T_SWITCH,
    Grid, harm_mean, iface_mean, make_env,
    interp_to_distances, save_result, TARGET_DISTS, params_at,
)

# =====
# 配置
# =====
N = 20                  # 径向网格数
DT = 0.5                # 时间步长 (s)
SAMPLE_DT = 60.0        # 输出采样间隔 (s)
T_MAX = 700000.0        # 仿真时长上限 (s), 约 194 h

grid = Grid(N)
Ta, Ca, _ = make_env()


# ==========
# 时间推进（含烘干完成事件检测）
# ==========
def solve():
    """返回 (时间数组, 水分浓度矩阵, t_f, C_at_tf, 守恒报告)。"""
    n = N + 1
    dr = grid.dr
    r_int, r_half = grid.r_int, grid.r_half
    A_R, A_in, V_h = grid.A_R, grid.A_inner, grid.V_half
    vols = grid.cell_volumes()

    C = np.full(n, C0)
    T = np.full(n, T0_C)

    water_init = float(np.sum(C * vols))
    evap_cum = 0.0

    times = [0.0]
    C_out = [C.copy()]
    next_sample = SAMPLE_DT
    stride = int(round(SAMPLE_DT / DT))

    t_f = None                  # 烘干完成时刻（插值到阈值穿越点）
    t_solved = None             # 严格达标的首个时间步
    C_at_tf = None              # 该时间步的水分浓度分布
    step = 0

    while step * DT < T_MAX and t_f is None:
        step += 1
        t_new = step * DT
        P = params_at(t_new - 0.5 * DT)

        # ---------- 物性（滞后取旧时刻值） ----------
        rho_n = P.rho(C)
        cp_n = P.cp(C)
        k_n = P.k(C)
        D_n = P.D(C, T + 273.15)

        k_h = harm_mean(k_n[:-1], k_n[1:])
        D_h = iface_mean(D_n[:-1], D_n[1:])   # 浓度连续，界面扩散系数取线性插值

        divT = np.zeros(n)
        divC = np.zeros(n)
```

```
        # 轴心：对称条件
        divT[0] = 4.0 * k_h[0] * (T[1] - T[0]) / dr ** 2
        divC[0] = 4.0 * D_h[0] * (C[1] - C[0]) / dr ** 2

        # 内部节点
        qT = r_half * k_h * (T[1:] - T[:-1]) / dr
        qC = r_half * D_h * (C[1:] - C[:-1]) / dr
        divT[1:N] = (qT[1:] - qT[:-1]) / (r_int * dr)
        divC[1:N] = (qC[1:] - qC[:-1]) / (r_int * dr)

        C_prev = C.copy()
        T_new = T + DT * divT / (rho_n * cp_n)
        C_new = np.concatenate([np.maximum(C[:N] + DT * divC[:N], 0.0), [C[N]]])

        # ---------------- 表面节点：半格控制体 + 隐式 ----------------
        Ta_v = Ta(t_new)
        rho_cp_s = rho_n[N] * cp_n[N]
        T_new[N] = ((rho_cp_s * V_h / DT * T[N]
                     + P.h * A_R * Ta_v
                     + k_h[-1] * A_in / dr * T_new[N - 1])
                    / (rho_cp_s * V_h / DT + P.h * A_R + k_h[-1] * A_in / dr))

        Ca_v = Ca(t_new)
        C_new[N] = ((V_h / DT * C[N]
                     + P.hm * A_R * Ca_v
                     + D_h[-1] * A_in / dr * C_new[N - 1])
                    / (V_h / DT + P.hm * A_R + D_h[-1] * A_in / dr))

        # 累计蒸发量 = 表面传质通量对时间积分（唯一的质量出口）
        evap_cum += P.hm * (C_new[N] - Ca_v) * A_R * DT
        T, C = T_new, C_new

        # ---------------- 烘干完成事件：中心首次跌破阈值 ----------------
        if t_f is None and C[0] <= C_SAFE:
            # 用线性插值定位穿越时刻，用于报告所需的精确时长
            c0, c1 = C_prev[0], C[0]
            frac = (c0 - C_SAFE) / (c0 - c1) if c0 != c1 else 0.0
            t_f = t_new - DT + DT * frac
            # 末行取越过阈值后 1 h 的状态，确保表中数值明显低于判据
            # （穿越瞬间 C 恰好等于阈值，四舍五入后仍显示 0.1500）
            extra = int(round(3600.0 / DT))
            for _ in range(extra):
                step += 1
                t_new = step * DT
                P = params_at(t_new - 0.5 * DT)
                rho_n, cp_n = P.rho(C), P.cp(C)
```

```
k_n, D_n = P.k(C), P.D(C
    ↪ , T + 273.15)
k_h = harm_mean(k_n
    ↪ [:-1], k_n[1:])
D_h = iface_mean(D_n
    ↪ [:-1], D_n[1:])
divT = np.zeros(n)
divC = np.zeros(n)
divT[0] = 4.0 * k_h[0] *
    ↪  (T[1] - T[0]) /
    ↪ dr ** 2
divC[0] = 4.0 * D_h[0] *
    ↪  (C[1] - C[0]) /
    ↪ dr ** 2
qT = r_half * k_h * (T
    ↪ [1:] - T[:-1]) /
    ↪ dr
qC = r_half * D_h * (C
    ↪ [1:] - C[:-1]) /
    ↪ dr
divT[1:N] = (qT[1:] - qT
    ↪ [:-1]) / (r_int *
    ↪  dr)
divC[1:N] = (qC[1:] - qC
    ↪ [:-1]) / (r_int *
    ↪  dr)
T_new = T + DT * divT /
    ↪ (rho_n * cp_n)
C_new = np.concatenate(
    [np.maximum(C[:N] +
        ↪ DT * divC[:N
        ↪ ], 0.0), [C[N
        ↪ ]]])
Ta_v, Ca_v = Ta(t_new),
    ↪ Ca(t_new)
rho_cp_s = rho_n[N] *
    ↪ cp_n[N]
T_new[N] = ((rho_cp_s *
    ↪ V_h / DT * T[N] +
    ↪  P.h * A_R * Ta_v
            + k_h[-1] *
                ↪
                ↪ A_in
                ↪  /
                ↪ dr *
                ↪
                ↪ T_new
                ↪ [N -
                ↪  1])
        / (rho_cp_s
            ↪ * V_h
            ↪  / DT
            ↪  + P.
            ↪ h *
            ↪ A_R
           + k_h[-1]
              ↪  *
              ↪
              ↪ A_in
              ↪  /
              ↪
              ↪ dr
              ↪ ))
C_new[N] = ((V_h / DT *
    ↪ C[N] + P.hm * A_R
    ↪  * Ca_v
            + D_h[-1] *
              ↪
              ↪ A_in
              ↪  /
              ↪ dr *
              ↪
              ↪ C_new
              ↪ [N -
              ↪  1])
        / (V_h / DT
            ↪ + P.
            ↪ hm *
            ↪ A_R +
            ↪  D_h
            ↪ [-1]
            ↪ *
            ↪ A_in
            ↪ / dr)
```

```
                                    )
                evap_cum += P.hm * (
                    C_new[N] - Ca_v)
                    * A_R * DT
                T, C = T_new, C_new
            C_at_tf = C.copy()
            t_solved = t_new
            break

        # ---------------- 采样 ----------------
        if step % stride == 0:
            times.append(t_new)
            C_out.append(C.copy())
            next_sample += SAMPLE_DT

    if t_f is None:
        print(f"[警告] 仿真达到上限 {T_MAX/3600:.1f} h，中心水分浓度仍为 {C[0]:.4f}")
        t_f = step * DT
        t_solved = t_f
        C_at_tf = C.copy()

    # 把「严格达标」时刻的真实状态追加为末行
    if times[-1] < t_solved:
        times.append(t_solved)
        C_out.append(C_at_tf.copy())

    # ---------------- 质量守恒校核 ----------------
    # 药材总水量的减少，应等于表面传质通量的时间积分（唯一质量出口）
    water_final = float(np.sum(C * vols))
    loss = water_init - water_final
    rel = (loss - evap_cum) / water_init * 100.0
    report = (f"初始水量 {water_init:.6e} 最终水量 {water_final:.6e}\n"
              f"内部失水量 {loss:.6e} 表面累计蒸发 {evap_cum:.6e}\n"
              f"守恒残差 {rel:.4f}%")

    return np.array(times), np.array(C_out), t_f, C_at_tf, report


# ======
# 主流程
# ======
times, C_out, t_f, C_at_tf, report = solve()

C_grid = np.array([interp_to_distances(grid.r, C_out[i], TARGET_DISTS)
                   for i in range(len(times))])
C_tf_grid = C_grid[-1]          # 末行即「严格达标」时刻的分布

print("=" * 60)
print("质量守恒")
print("=" * 60)
print(report)

save_result(
    os.path.join(BASE_DIR, "result3.xlsx"),
    [("水分浓度", times, C_grid, "时间\\到药材中心的距离")],
)

# ========
# 表5
# ========
TABLE_HOURS = [6, 12, 18, 24, 30, 36, 42]
TABLE_DISTS = [0.0, 0.5, 1.0, 1.5, 2.0]
```

```
def col(d):
    return int(np.argmin(np.abs(
        ↪ TARGET_DISTS - d)))


def row(t_val):
    return int(np.argmin(np.abs(times -
        ↪ t_val)))


print("\n" + "=" * 72)
print("表5  药材烘干过程的水分浓度 (kg/
    ↪ kg)")
print(f"烘干结束时间: {t_f/3600:.2f} h
    ↪ ({t_f:.0f} s)")
print("=" * 72)
print(f"{'时间/h':<12}" + "".join(f"{d}
    ↪ cm".rjust(12) for d in
    ↪ TABLE_DISTS))
print("-" * 72)
for th in TABLE_HOURS:
    t_val = min(int(th * 3600), int(
        ↪ times[-1]))
    print(f"{th:<12.1f}" + "".join(f"{
        ↪ C_grid[row(t_val), col(d)
        ↪ ]:>12.4f}" for d in
        ↪ TABLE_DISTS))
print(f"{t_f/3600:<12.2f}" + "".join(f"{
    ↪ C_tf_grid[col(d)]:>12.4f}" for d
    ↪ in TABLE_DISTS))
print(f"\n[环境] 阶段切换 t_c = {
    ↪ T_SWITCH/3600:.1f} h; 物性保护下
    ↪ 界 C_CLIP_LO = {C_CLIP_LO:.0e}")
print(f"[判据] 中心 C(0,t_f) = {C_grid
    ↪ [-1, col(0.0)]:.4f} kg/kg < {
    ↪ C_SAFE}, "
      f"各处最大 C = {C_grid[-1].max()
          ↪ :.4f} kg/kg")
print(f"[输出] result3.xlsx: {len(times)
    ↪ } 行 x {len(TARGET_DISTS)} 列")
```

Listing 3: 主程序代码

```
"""
问题4: 变尺寸 (移动边界) 条件下的烘干时
    ↪ 长。

模型: 与问题2/3 同一套圆柱双场耦合, 改写
    ↪ 到 Landau (固连) 坐标   = r/R(t)
    ↪ :
     C/ t|       = (1/(R(t)² · ))  /   (
        ↪   D(C,T)  C/   )
     c_p  T/ t|  = (1/(R(t)² · ))  /   (
        ↪   k(C)  T/   )
干基口径下方程不含体积收缩修正项: 干基含
    ↪ 水率的分母 (干料质量) 在收缩中
守恒, 固相连坐标的物质导数自动吸收固相速
    ↪ 度项, 方程与固定域形式同构,
仅 r → R(t)· , 且方程中只出现 R(t) 不出
    ↪ 现 Ṙ。

物性: 附录4 全套 (问题4 专用):
      = 760 + 90C; c_p = 1850 + 2150C/(C
        ↪ +1); k = 0.12 + 0.20C/(C+1)
    D = 4.2e-4 · exp(-0.30/C) · exp
        ↪ (-3850/T)    (T 单位 K)
几何: R(t) 由附件2 实测给出 (2.0 → 1.198
    ↪  cm), PCHIP 插值, 超出末端取末
    ↪ 值。
环境: 与问题2/3 完全一致 (make_env: 预热
    ↪ 平衡实测 + 恒温干燥外推)。

边界:   = 0 对称 (轴心 4× 格式);   = 1
    ↪ 表面 Robin (半格控制体隐式):
    传质  -D  C/ r = h_m (C_a - C_s)   →
        ↪      形式对流系数 h_m/R(t)
    换热  -k  T/ r = h   (T_a - T_s)   →
        ↪      形式对流系数 R(t)·h
    界面导热/扩散系数只依赖   (Landau 红
        ↪ 利:  _{N-1/2}·k/Δ  与 R 无
        ↪ 关)。
    R   R_0 时全部系数逐项退化为问题2/3
        ↪ 的固定域格式。
```

```
格式：内部节点显式，表面半格节点隐式；默认 Δ = 0.05（21 节点，恰为初始半径
    0.1 cm 间隔），Δt = 0.5 s。稳定性（R 取最小 1.198 cm）：
    4·D·Δt/(RΔ)²  0.01（水分场）、4· ·Δt/(RΔ)²  0.49（温度场）。

判据：中心  = 0（最不利点）水分浓度首次降至 C_SAFE = 0.15 kg/kg，
    线性插值定位穿越时刻；出表末行取严格达标后再推进 1 h 的状态。

算例（run_case）：
    q4    附录4 全套物性 + 移动边界 R(t)  —— 生产算例
    regr  附录3 物性 + 固定几何 R  R_0  —— 回归验证，应复现 solve_problem3.py
    fix4  附录4 物性 + 固定几何 R  R_0  —— 归因分解（隔离收缩效应）
    另可用 N / DT / clip_lo 参数做网格收敛与物性下界双算例。

输出 result4.xlsx：
    「水分浓度」表 —— A 列时间(s)；B~V 列为材料坐标（初始半径）
    0.0, 0.1, …, 1.9 cm 与末列「药材表面」（=  =1，即当前表面）。
    每列固定追踪同一物质点（其物理半径 = 列名 × R(t)/R_0），
    与模板「0 | 0.1 | … | 药材表面」的 0.1 cm 间隔一致。
    「半径」表 —— 各采样时刻的 R(t)（cm），供查表与作图。

命令行：
    python solve_problem4.py [q4|regr|fix4]
"""

import os
import sys

import numpy as np
import pandas as pd
from scipy.interpolate import PchipInterpolator

from dry_model import (
    C0, T0_C, C_SAFE, C_CLIP_LO,
        C_CLIP_HI, BASE_DIR,
    R as R0, make_env, locate, harm_mean
        , iface_mean,
    rho3, cp3, k3, D3,
)

# ========
# 常量配置
# ========
DT_DEFAULT = 0.5       # 时间步长（s）
N_DEFAULT = 20         #  方向网格数（Δ = 0.05, 21 节点）
SAMPLE_DT = 60.0       # 输出采样间隔（s）
T_MAX = 700000.0       # 仿真时长上限（s），约 194 h

H_CONV = 25.0          # 对流换热系数 W/(m²·K)（附录2，与问题1-3 一致）
HM_CONV = 8e-7         # 对流传质系数 m/s（附录2，与问题1-3 一致）


# ===========
# 附录4 物性（工厂：clip_lo 可调，用于"夹紧 vs 自由外推"双算例）
# ===========
def make_props4(clip_lo=C_CLIP_LO):
    """返回附录4 物性函数组（rho, cp, k, D）；clip_lo 为浓度数值下界。"""
    def _c(C):
```

```
        return np.clip(C, clip_lo, C_CLIP_HI)

    def rho(C):
        return 760.0 + 90.0 * _c(C)

    def cp(C):
        Cp = _c(C)
        return 1850.0 + 2150.0 * Cp / (Cp + 1.0)

    def k(C):
        Cp = _c(C)
        return 0.12 + 0.20 * Cp / (Cp + 1.0)

    def D(C, T_K):
        return 4.2e-4 * np.exp(-0.30 / _c(C)) \
            * np.exp(-3850.0 / np.clip(T_K, 250.0, 400.0))

    return rho, cp, k, D


rho4, cp4, k4, D4 = make_props4(C_CLIP_LO)      # 默认物性（与问题3 的解耦口径一致）


# =============
# 几何：附件2 的 R(t)
# =============
def load_radius():
    """读取附件2（时间 s，半径 cm），返回 R_of(t)（单位 m）。

    PCHIP 保单调、无过冲；超出数据末端（259200 s = 72 h）取末值 1.198 cm。
    """
    df = pd.read_excel(locate("附件2.xlsx"))
    t_d = df["时间"].values.astype(float)
    r_d = df["半径"].values.astype(float) * 1e-2      # cm → m
    pR = PchipInterpolator(t_d, r_d, extrapolate=False)

    def R_of(t):
        return float(pR(np.clip(t, t_d[0], t_d[-1])))

    return R_of


# 环境函数（所有算例共用，与问题2/3 完全一致）
TA_FUN, CA_FUN, _ = make_env()


# =============
# 通用算例求解器
# =============
def run_case(mode="q4", N=N_DEFAULT, DT=DT_DEFAULT, clip_lo=C_CLIP_LO,
             sample_dt=SAMPLE_DT, t_max=T_MAX, verbose=False):
    """求解一个算例。

    mode: "q4"   附录4 + 移动边界 R(t)
          "regr" 附录3 + 固定几何 R   R0（回归验证）
          "fix4" 附录4 + 固定几何 R   R0（归因分解）

    返回 dict：times, C(采样矩阵), R(采样半径数组 m), xi(节点  ),
                t_f, t_s, t_solved, C_tf(达标后 1 h 的节点浓度),
                T_max, report(质量守恒报
```

```
                ↪ 告), N, DT
    """
    if mode == "regr":
        rho_f, cp_f, k_f, D_f = rho3,
            ↪ cp3, k3, D3
        R_of = lambda t: R0
    elif mode == "fix4":
        rho_f, cp_f, k_f, D_f =
            ↪ make_props4(clip_lo)
        R_of = lambda t: R0
    elif mode == "q4":
        rho_f, cp_f, k_f, D_f =
            ↪ make_props4(clip_lo)
        R_of = load_radius()
    else:
        raise ValueError(f"未知算例模式
            ↪ {mode!r}")

    # ---------------- Landau (固连) 网
        ↪ 格 ----------------
    n = N + 1
    dxi = 1.0 / N
    xi = np.linspace(0.0, 1.0, n)
        ↪                          # 节点
        ↪   _j = j·Δ
    xi_half = (np.arange(N) + 0.5) * dxi
        ↪                    # 界面  _{j
        ↪ +1/2}
    bnd = np.concatenate(([0.0], xi_half
        ↪ , [1.0]))
    W = 0.5 * (bnd[1:] ** 2 - bnd[:-1]
        ↪ ** 2)              # 控制体
        ↪ 测度   d  (不随时间变)
    assert abs(W.sum() - 0.5) < 1e-12

    C = np.full(n, C0)
    T = np.full(n, T0_C)

    # ---------------- 显式格式稳定性预
        ↪ 检 ----------------
    # 内部节点显式推进要求 4· ·Δt/(
        ↪ R_min·Δ)²   1 (轴心节点最
        ↪ 严);
    # 网格加密时 Δt 必须按 Δ² 同步缩
        ↪ 小, 否则会发散 (NaN)。
    _Cg = np.linspace(max(clip_lo, 1e-3)
        ↪ , C0, 64)
    alpha_max = float(np.max(k_f(_Cg) /
        ↪ (rho_f(_Cg) * cp_f(_Cg))))
    R_min = min(R_of(0.0), R_of(1e9))
    stab = 4.0 * alpha_max * DT / (R_min
        ↪  * dxi) ** 2
    if stab > 1.0:
        raise ValueError(
            f"显式格式不稳定: 4· ·Δt/(
                ↪ R·Δ)² = {stab:.3f} >
                ↪  1 (N={N}, Δt={DT}
                ↪ )。"
            f"请将 Δt 减小到 {DT / stab
                ↪ :.3f} s 以下 (Δt 需
                ↪  Δ²)。")
    if verbose:
        print(f"[稳定性] 4·max·Δt/(R·Δ
            ↪ )² = {stab:.4f}   1  "
              f"( max = {alpha_max:.3e}
                  ↪ m²/s, R_min = {
                  ↪ R_min*100:.3f} cm)"
                  ↪ )

    water_init = float(np.sum(C * W))
        ↪      # 干基口径总水量 (  物理
        ↪ 水量, 权重恒定)
    evap_cum = 0.0

    times = [0.0]
    C_out = [C.copy()]
    R_out = [R_of(0.0)]
    stride = max(1, int(round(sample_dt
        ↪ / DT)))

    t_f = None                  # 中心首次
        ↪ 达标时刻 (插值)
    t_s = None                  # 表面首次
        ↪ 达标时刻 (插值)
```

```
    t_solved = None            # 严格达标后再推进 1 h 的末行时刻
    C_at_tf = None
    T_max = T0_C
    step = 0

    def fields(C, T, t_new):
        """推进一个时间步：内部显式 + 表面半格隐式（问题2/3 的形式）。"""
        R_t = R_of(t_new)
        R2 = R_t * R_t

        # ---------- 物性（滞后取旧时刻值）----------
        rho_n = rho_f(C)
        cp_n = cp_f(C)
        k_n = k_f(C)
        D_n = D_f(C, T + 273.15)
        k_h = harm_mean(k_n[:-1], k_n[1:])
        D_h = iface_mean(D_n[:-1], D_n[1:])   # 浓度连续，界面扩散系数取线性插值

        # 界面通量（ 形式）：q = ·coef·Δfield/Δ ；R 的作用集中在 1/R²
        qC = xi_half * D_h * (C[1:] - C[:-1]) / dxi
        qT = xi_half * k_h * (T[1:] - T[:-1]) / dxi

        divC = np.zeros(n)
        divT = np.zeros(n)
        # 轴心：对称条件（等价于 4×(field[1]-field[0]) 格式）
        divC[0] = qC[0] / (R2 * W[0])
        divT[0] = qT[0] / (R2 * W[0])
        # 内部节点
        divC[1:N] = (qC[1:] - qC[:-1]) / (R2 * W[1:N])
        divT[1:N] = (qT[1:] - qT[:-1]) / (R2 * W[1:N])

        # ---------- 内部节点推进（显式）----------
        T_new = T + DT * divT / (rho_n * cp_n)
        C_new = np.concatenate([np.maximum(C[:N] + DT * divC[:N], 0.0), [C[N]]])

        # ---------- 表面节点：半格控制体 + 隐式 ----------
        xi_s = xi_half[-1]                    # 1 - Δ/2，界面系数与 R 无关
        Ta_v, Ca_v = TA_FUN(t_new), CA_FUN(t_new)

        # 能量：R²·w·cp·(T_N^n - T_N)/Δt = _s·k/Δ·(T_{N-1}^n - T_N^n) + R·h·(Ta - T_N^n)
        rho_cp_s = rho_n[N] * cp_n[N]
        T_new[N] = ((R2 * W[N] * rho_cp_s / DT * T[N]
                     + xi_s * k_h[-1] / dxi * T_new[N - 1]
                     + R_t * H_CONV * Ta_v)
                    / (R2 * W[N] * rho_cp_s / DT
                       + xi_s * k_h[-1] / dxi + R_t * H_CONV))

        # 水分：w·(C_N^n - C_N)/Δt = (h_m/R)·(Ca - C_N^n) +
```

```
            ↪ _s · D/(Δ R²) · (C_{N-1}^n −
            ↪  C_N^n)
        C_new[N] = ((W[N] / DT * C[N]
                     + HM_CONV / R_t *
                         ↪ Ca_v
                     + xi_s * D_h[-1] /
                         ↪ (dxi * R2) *
                         ↪  C_new[N -
                         ↪ 1])
                    / (W[N] / DT +
                        ↪ HM_CONV / R_t
                       + xi_s * D_h[-1]
                           ↪ / (dxi *
                           ↪ R2)))

        # 边界蒸发量（干基口径）：d(ΣWC)
            ↪ /dt = -(h_m/R)·(C_N - Ca)
        evap = HM_CONV / R_t * (C_new[N]
            ↪  - Ca_v) * DT
        return C_new, T_new, evap

    while step * DT < t_max and t_f is
        ↪ None:
        step += 1
        t_new = step * DT
        C_prev = C.copy()

        C, T, evap = fields(C, T, t_new)
        evap_cum += evap
        T_max = max(T_max, float(T.max()
            ↪ ))

        # ---------------- 表面首次达标
            ↪ ----------------
        if t_s is None and C[N] <=
            ↪ C_SAFE:
            frac = ((C_prev[N] - C_SAFE)
                ↪  / (C_prev[N] - C[N])
                    if C_prev[N] != C[N]
                        ↪  else 0.0)
            t_s = t_new - DT + DT * frac

        # ---------------- 中心首次达
            ↪ 标：线性插值 + 延长 1 h
            ↪ ----------------
        if C[0] <= C_SAFE:
            frac = ((C_prev[0] - C_SAFE)
                ↪  / (C_prev[0] - C[0])
                    if C_prev[0] != C[0]
                        ↪  else 0.0)
            t_f = t_new - DT + DT * frac
            extra = int(round(3600.0 /
                ↪ DT))
            for _ in range(extra):
                step += 1
                t_new = step * DT
                C, T, evap = fields(C, T
                    ↪ , t_new)
                evap_cum += evap
                T_max = max(T_max, float
                    ↪ (T.max()))
            C_at_tf = C.copy()
            t_solved = t_new
            break

        # ---------------- 采样
            ↪ ----------------
        if step % stride == 0:
            times.append(t_new)
            C_out.append(C.copy())
            R_out.append(R_of(t_new))

    if t_f is None:
        print(f"[警告] 仿真达到上限 {
            ↪ t_max/3600:.1f} h，中心水
            ↪ 分浓度仍为 {C[0]:.4f}")
        t_f = step * DT
        t_solved = t_f
        C_at_tf = C.copy()

    # 把「严格达标 + 1 h」时刻的真实状态
        ↪ 追加为末行
    if times[-1] < t_solved:
        times.append(t_solved)
```

```
        C_out.append(C_at_tf.copy())
        R_out.append(R_of(t_solved))

    # ---------------- 质量守恒校核（干基口径） ----------------
    # 药材总水量（ΣWC，权重恒定）的减少应等于表面传质通量的时间积分
    water_final = float(np.sum(C * W))
    loss = water_init - water_final
    rel = (loss - evap_cum) / water_init * 100.0
    report = (f"初始水量 {water_init:.6e} 最终水量 {water_final:.6e}\n"
              f"内部失水量 {loss:.6e} 表面累计蒸发 {evap_cum:.6e}\n"
              f"守恒残差 {rel:.4f}%")

    return {
        "mode": mode, "N": N, "DT": DT, "clip_lo": clip_lo,
        "xi": xi, "W": W,
        "times": np.array(times),
        "C": np.array(C_out),
        "R": np.array(R_out),
        "t_f": t_f, "t_s": t_s, "t_solved": t_solved,
        "C_tf": C_at_tf, "T_max": T_max, "report": report,
        "rel": rel,
    }


# =============
# 主流程（命令行）
# =============
def main():
    mode = sys.argv[1].strip().lower() if len(sys.argv) > 1 else "q4"
    if mode not in ("q4", "regr", "fix4"):
        raise SystemExit(f"用法：python solve_problem4.py [q4|regr|fix4]，未知模式 {mode!r}")

if mode == "q4":
    print("[模式] Q4：附录4 全套物性 + 附件2 移动边界 R(t)")
elif mode == "regr":
    print("[模式] REGR 回归验证：R = R0 恒定 + 附录3 物性 —— 应复现 solve_problem3.py")
else:
    print("[模式] FIX4 归因分解：附录4 物性 + 固定几何 R = R0")

res = run_case(mode, verbose=True)

N, DT = res["N"], res["DT"]
xi, times = res["xi"], res["times"]
C_out, R_out = res["C"], res["R"]
t_f, t_s, t_solved = res["t_f"], res["t_s"], res["t_solved"]
C_at_tf, T_max = res["C_tf"], res["T_max"]

# ---------------- 输出文件 ----------------
if mode == "q4":
    out_path = os.path.join(BASE_DIR, "result4.xlsx")
elif mode == "regr":
    out_path = os.path.join(BASE_DIR, "result4_REGR.xlsx")
else:
    out_path = os.path.join(BASE_DIR, "result4_FIX4.xlsx")

# 节点 j 的材料坐标 = _j · R0（N=20
```

```
        ↪ 时即 0.1j cm）；j = N 即「药
        ↪ 材表面」
    cols = [f"{xi[j] * R0 * 100:.1f}"
        ↪ for j in range(N)] + ["药材表
        ↪ 面"]
    df_C = pd.DataFrame(np.round(C_out,
        ↪ 4), columns=cols)
    df_C.insert(0, "时间\\到药材中心的距
        ↪ 离", times.astype(int))
    df_R = pd.DataFrame({
        "时间": times.astype(int),
        "半径": np.round(R_out * 100.0,
            ↪ 4),        # cm
    })
    with pd.ExcelWriter(out_path, engine
        ↪ ="openpyxl") as writer:
        df_C.to_excel(writer, sheet_name
            ↪ ="水分浓度", index=False)
        df_R.to_excel(writer, sheet_name
            ↪ ="半径", index=False)

    # ---------------- 物理自检
        ↪ ----------------
    mono = bool(np.all(np.diff(C_out,
        ↪ axis=0) <= 1e-9))    # 无水源
        ↪ → 单调不增
    C_avg_tf = float(np.sum(res["W"] *
        ↪ C_at_tf) / res["W"].sum())

    print("=" * 60)
    print("物理自检")
    print("=" * 60)
    print(f"浓度单调不增（每节点）：  {'
        ↪ 通过' if mono else '失败'}")
    print(f"温度上界 T_max = {T_max:.3f}
        ↪ ℃（不应超过环境最高约 50.25
        ↪ ℃）")
    print(f"t_f 时刻面积加权平均含水率 =
        ↪  {C_avg_tf:.4f} kg/kg")
    print()
    print("=" * 60)
    print("质量守恒（干基口径）")
    print("=" * 60)
    print(res["report"])

    # ---------------- 表6：每 6 h、材料
        ↪ 坐标 0 / 0.5 / 1.0 / 1.5 cm
        ↪ 与药材表面 ----------------
    idx = [int(round(f * N)) for f in
        ↪ (0.0, 0.25, 0.5, 0.75, 1.0)]
        ↪   #   = 0,0.25,0.5,0.75,1
    lbl = [f"{xi[j] * R0 * 100:.1f}" for
        ↪  j in idx[:-1]] + ["表面"]

    def row_of(t_val):
        return int(np.argmin(np.abs(
            ↪ times - t_val)))

    print()
    print("=" * 72)
    print("表6  药材烘干过程的水分浓度（
        ↪ kg/kg）  [列 = 材料坐标，末列
        ↪  = 药材表面]")
    print(f"烘干结束时间：{t_f/3600:.2f}
        ↪  h ({t_f:.0f} s)")
    print("=" * 72)
    print(f"{'时间/h':<12}" + "".join(f"
        ↪ {l}cm".rjust(12) for l in lbl
        ↪ ))
    print("-" * 72)
    for th in [6, 12, 18, 24, 30, 36,
        ↪ 42]:
        if th * 3600 > times[-1]:
            continue
        print(f"{th:<12.1f}" + "".join(f
            ↪ "{C_out[row_of(th*3600),
            ↪ j]:>12.4f}" for j in idx)
            ↪ )
    print(f"{t_f/3600:<12.2f}" + "".join
        ↪ (f"{C_at_tf[j]:>12.4f}" for j
        ↪  in idx))

    # ---------------- 结果摘要
        ↪ ----------------
```

```
    print()
    print("=" * 60)
    print("结果摘要")
    print("=" * 60)
    print(f"烘干总时长 t_f = {t_f /3600:.4f} h = {t_f:.1f} s {t_f/86400:.2f} 天")
    if t_s is not None:
        print(f"表面首次达标 t_s = {t_s /3600:.4f} h；表面→中心时滞 "
              f"{(t_f - t_s)/3600:.2f} h ")
    print(f"t_f 时刻半径 R(t_f) = {R_out[-1] * 100:.4f} cm")
    print(f"[判据] 中心 C(0, t_f) = {C_at_tf[0]:.4f} kg/kg  {C_SAFE}, "
          f"各处最大 C = {C_at_tf.max():.4f} kg/kg")
    print(f"[输出] {os.path.basename(out_path)}: {len(times)} 行 x {len(cols)} 列"
          f" (+ 半径表 {len(times)} 行)")
    if mode == "regr":
        print("[核对] 请与 solve_problem3.py 的 t_f 及 result3.xlsx 末行时间比对")


if __name__ == "__main__":
    main()
```

Listing 4: 主程序代码